



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:
«Плагин для VS-Code для автоматизации
формирования Unit тестов кроссплатформенных
приложений на Flutter»

Студент _____
(Группа)

(Подпись, дата)

(И.О. Фамилия)

Руководитель ВКР

(Подпись, дата)

(И.О. Фамилия)

Консультант

(Подпись, дата)

(И.О. Фамилия)

Консультант

(Подпись, дата)

(И.О. Фамилия)

Нормоконтролер

(Подпись, дата)

(И.О. Фамилия)

2025 г.

АННОТАЦИЯ

Работа посвящена разработке плагина для интегрированной среды разработки Visual Studio Code, предназначенного для автоматической генерации модульных тестов для кроссплатформенных приложений на основе фреймворка Flutter. Рассматриваются подходы к генерации тестов для компонентов управления состоянием Cubit/BLoC с использованием моделей конечных автоматов и теории тестовых множеств, а также для DTO(data transfer object) и обычных методов/функций на языке Dart. Описываются архитектура плагина, процесс парсинга исходного кода, алгоритмы генерации тестовых сценариев и интеграция с инструментами тестирования Flutter, такими как пакет `bloc_test`.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 Исследовательская часть	6
1.1 Постановка задачи и техническое задание	6
1.1.1 Требования к интеграции в IDE	6
1.1.2 Требования к генерации тестовых файлов	6
1.1.3 Требования к генерации тестов для различных компонентов	7
1.2 Статические методы генерации тестов	8
1.3 Теоретические основы тестирования на основе моделей	10
1.3.1 Конечные автоматы как модель поведения	10
1.3.2 Тестовые множества и полнота тестирования	11
1.3.3 Цели тестового покрытия для автоматов	12
1.4 Генерация тестов для Cubit с использованием автоматов	13
1.4.1 Построение автомата состояний	13
1.4.2 Генерация тестовых путей	14
1.4.3 Трансляция путей в тестовый код	16
1.5 Анализ подхода и анализ релевантности	17
1.5.1 Преимущества	17
1.5.2 Ограничения и сложности	17
1.5.3 Анализ релевантности	17
1.6 Обзор предметной области	19
1.6.1 Используемые технологии	19
1.6.2 Тестирование в разработке на Flutter	21
1.6.3 Проблемы ручного тестирования и необходимость автоматизации	21
1.6.4 Постановка задачи разработки плагина	21
2 Конструкторская часть	23
2.1 Проектирование плагина	23
2.1.1 Архитектура плагина	23
2.1.2 Основные интерфейсы и структуры данных	24
2.1.3 Основные функциональные компоненты	30
2.1.4 Порядок выполнения	32

2.1.5	Взаимодействие с пользователем	32
2.1.6	Парсинг исходного кода	33
3	Технологическая часть	35
3.1	Реализация ключевых функций	35
3.1.1	Генерация тестов для Cubit/BLoC	35
3.1.2	Генерация тестов для Data Transfer Object	36
3.1.3	Генерация тестов для методов/ивентов	37
3.1.4	Решение проблем и доработки	38
3.2	Инструкция пользователя	39
4	Аналитическая часть	42
4.1	Результаты работы	42
4.1.1	Результаты генерации тестов для Cubit	42
4.1.2	Результаты генерации тестов для BLoC	42
4.1.3	Результаты генерации тестов для Data Transfer Object	43
4.1.4	Качественные характеристики генерируемых тестов	43
4.1.5	Практическая ценность результатов	44
4.2	Анализ эффективности применения плагина	44
4.2.1	Методология оценки	44
4.2.2	Временные характеристики	45
4.2.3	Сводный анализ эффективности	47
4.2.4	Качественный анализ	48
4.2.5	Выводы по анализу эффективности	48
	ЗАКЛЮЧЕНИЕ	50
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	52
	ПРИЛОЖЕНИЕ А	52

ВВЕДЕНИЕ

Развитие мобильных и веб-приложений с использованием кроссплатформенных фреймворков, таких как Flutter [**FlutterDev**], ставит перед разработчиками задачи не только быстрого создания функциональности, но и обеспечения высокого качества программного продукта. Модульное тестирование является ключевым элементом этого процесса [**FlutterTestingCookbook**], позволяя выявлять ошибки на ранних этапах и обеспечивать стабильность кодовой базы. Однако ручное написание тестов, особенно для компонентов со сложной логикой управления состоянием (например, Cubit в экосистеме BLoC [**BlocLibraryTesting**]), является трудоемким и подверженным ошибкам процессом.

Актуальность данной работы обусловлена необходимостью снижения трудозатрат на написание модульных тестов и повышения их полноты. Автоматизация генерации тестов позволяет разработчикам сосредоточиться на более сложных аспектах проектирования и реализации, при этом получая базовое тестовое покрытие для своих компонентов.

Целью настоящей работы является разработка плагина для среды VS Code [**VSCoDeAPI**], который автоматизирует процесс создания модульных тестов для Flutter-приложений [**FlutterUnitTesting**], в частности для классов Cubit/BLoC, классов DTO и методов.

Основные задачи включают:

- Анализ структуры Dart-кода [**DartDev**] для извлечения информации о классах Cubit/BLoC, их состояниях, ивентах и методах, изменяющих состояние.
- Разработку алгоритма построения модели поведения компонента Cubit, приближенной к модели конечного автомата.
- Реализацию механизма генерации тестовых сценариев на основе анализа возможных переходов состояний Cubit/BLoC с использованием пакета `bloc_test` [**BlocTestPackage**].
- Обеспечение генерации шаблонов тестов для отдельных методов/ивентов классов.
- Интеграцию плагина в среду разработки VS Code для удобного использования разработчиками.

1 Исследовательская часть

В данной главе описаны основные теоретические моменты, которые прямо или косвенно используются в дипломной работе. Описаны ключевые требования и теоремы теории формальных языков.

Необходимо разработать и описать систему автоматической генерации модульных тестов, интегрируемую в среду разработки (IDE). Система должна анализировать выбранный пользователем программный компонент (метод, DTO, Cubit) и генерировать для него тестовый файл с набором тестовых случаев.

1.1 Постановка задачи и техническое задание

1.1.1 Требования к интеграции в IDE

Ожидаемое поведение IDE:

1. Система должна активироваться по специальной команде (например, сочетание клавиш `cmd+ .` или аналогичное) при установке курсора на имени метода, класса DTO или класса Cubit в редакторе кода (см. документацию VS Code API [VSCodeAPI]).
2. В контекстном меню, появляющемся после вызова команды, должен присутствовать пункт «Generate test» (или локализованный аналог).
3. При выборе пункта «Generate test» система должна автоматически сгенерировать или обновить тестовый файл.

1.1.2 Требования к генерации тестовых файлов

Ожидаемая структура тестовой директории:

1. Тестовый файл должен располагаться в стандартной директории для тестов проекта (например, `test/`) во вложенной директории, соответствующей «фиче» (feature) или модулю, к которому относится тестируемый компонент. Путь должен формироваться автоматически на основе пути к исходному файлу компонента. Пример: для компонента в файле `lib/features/user_profile/data/models/user_dto.dart` тест должен генерироваться в

test/features/user_profile/data/models/user_dto_test.dart. 2. Имя тестового файла должно формироваться из имени исходного файла с добавлением суффикса `_test` (например, `user_dto.dart` -> `user_dto_test.dart`). 3. Генерируемый тестовый файл должен содержать максимально полный набор тестовых случаев, который возможно создать автоматически на основе анализа логики и структуры тестируемого компонента. 4. Если тестовый файл уже существует, плагин должен создать альтернативный файл с префиксом `_gen`, используя стандартные возможности фреймворка тестирования Dart [**DartTestPackage**].

1.1.3 Требования к генерации тестов для различных компонентов

Методы

Тесты для методов должны генерироваться статически. Исходя из простоты тела теста методов, применение логики, заимствованной из теории формальных языков, является излишней, поэтому плагин генерирует примитивный шаблон, который пользователь наполняет ожидаемыми значениями. Пример заглушки теста для метода приведен в Листинге 1 Приложения А.

Объекты передачи данных (DTO)

Тесты для DTO должны генерироваться статически и проверять:

- Корректность создания экземпляра DTO с валидными данными.
- Корректность работы методов сериализации/десериализации (например, `toJson/fromJson`), если они присутствуют.

Пример заглушки теста для DTO приведен в Листинге 2 Приложения А.

Компоненты управления состоянием (Cubit)

Тесты для Cubit должны генерироваться динамически с использованием модели конечного автомата.

1. Система должна построить модель конечного автомата, где:

- Узлы (состояния) автомата соответствуют возможным классам состояний (State) данного Cubit.
- Переходы (ребра) автомата соответствуют публичным методам Cubit, которые потенциально изменяют его состояние. Метка перехода включает имя метода и, возможно, типы его аргументов.

2. На основе построенного автомата система должна сгенерировать тестовые сценарии.
3. Каждый тестовый сценарий соответствует некоторому пути в автомате.
4. Система должна сгенерировать до 10 различных тестовых сценариев (путей), если такое количество возможно. Выбор путей должен стремиться к покрытию различных состояний и переходов.
5. Каждый сгенерированный тест должен выполнять последовательность действий:
 - Инициализировать Cubit в начальном состоянии пути.
 - Последовательно вызывать методы Cubit, соответствующие переходам на пути.
 - После каждого вызова проверять (assert), что Cubit перешел в ожидаемое состояние, соответствующее следующему узлу на пути.

Пример заглушки теста для Cubit, сгенерированного на основе пути автомата, приведен в Листинге 5 Приложения А. Пример графа автомата для гипотетического Cubit показан на Рисунке ?? Приложения Б.

1.2 Статические методы генерации тестов

Статическая генерация тестов подходит для компонентов с относительно простой структурой и поведением, где основные проверки можно вывести непосредственно из сигнатур методов, типов данных и общепринятых соглашений. К таким компонентам относятся простые функции/методы и объекты передачи данных (DTO).

Генерация тестов для методов

Для генерации тестов методов без состояния или сложной логики можно использовать анализ их сигнатур.

1. Анализ аргументов: Определяются типы и имена аргументов. Для примитивных типов (числа, строки, булевы значения) можно сгенерировать тесты с типичными и граничными значениями: — Числа: 0, 1, -1, большое положительное, большое отрицательное. — Строки: пустая строка, строка с пробелами, строка с не-ASCII символами, null (если допустимо). — Булевы значения: true, false. — Коллекции: пустая коллекция, коллекция с одним элементом, коллек-

ция с несколькими элементами, `null` (если допустимо). 2. Анализ возвращаемого значения: Генерируется проверка на соответствие типа возвращаемого значения ожидаемому типу. Если метод должен возвращать конкретное значение при определенных входных данных (что сложно определить статически без семантического анализа), генерируется заглушка теста с комментарием о необходимости указать ожидаемый результат. 3. Обработка исключений: Если сигнатура метода объявляет возможные исключения (например, через аннотации или ключевые слова типа `throws`), можно сгенерировать тесты, проверяющие выброс этих исключений при соответствующих условиях (например, при передаче некорректных аргументов). 4. Мокирование зависимостей: Если метод принимает в качестве аргументов объекты других классов (зависимости), система может попытаться сгенерировать код для создания мок-объектов (`mock objects`) с использованием популярных фреймворков (например, Mockito [**MockitoDart**], Mocktail). Генерируются базовые настройки моков (например, возврат `null` или значений по умолчанию).

Статическая генерация для методов обычно создает каркас теста, который разработчику часто требуется доработать, указав конкретные ожидаемые результаты или настроив мок-объекты более детально. Пример такого каркаса представлен в Приложении А (Листинг 1).

Генерация тестов для Data Transfer Object

Объекты передачи данных (DTO) обычно представляют собой простые структуры для хранения данных с минимальной логикой. Тесты для них направлены на проверку корректности хранения, доступа, сериализации и сравнения данных.

1. Тестирование конструктора/фабрики: Генерируется тест, создающий экземпляр DTO с использованием его конструктора или статического фабричного метода (если таковой имеется) с набором валидных данных. Проверяется, что все поля объекта инициализированы корректно. 2. Тестирование сериализации/десериализации: Если DTO содержит методы `toJson` и `fromJson` (или аналогичные), генерируются тесты для них: — `toJson`: Создается экземпляр DTO, вызывается `toJson`, проверяется, что результат является ожидаемой JSON-структурой (`Map<String, dynamic>` или аналогичной). — `fromJson`: Создается тестовая JSON-структура, вызывается `fromJson`, проверяется, что созданный экземпляр

DTO содержит ожидаемые данные. Система может попытаться сгенерировать примеры JSON-структур на основе полей DTO.

Пример теста для DTO приведен в Приложении А (Листинг 2). Статическая генерация для DTO часто позволяет создать достаточно полные и готовые к использованию тесты.

1.3 Теоретические основы тестирования на основе моделей

1.3.1 Конечные автоматы как модель поведения

Тестирование на основе моделей (Model-Based Testing, MBT) — это подход к тестированию программного обеспечения, при котором тестовые сценарии выводятся автоматически из модели, описывающей поведение системы или ее компонента. Для компонентов с состоянием, таких как Cubit, в качестве модели удобно использовать конечные автоматы.

Определение 1 (Конечный автомат). *Детерминированный конечный автомат (ДКА) [AhoHopcroftUllman1974] — это кортеж $M = (Q, \Sigma, \delta, q_0, F)$, где:*

- Q — конечное множество состояний.
- Σ — конечный входной алфавит (множество символов или событий).
- $\delta : Q \times \Sigma \rightarrow Q$ — функция переходов.
- $q_0 \in Q$ — начальное состояние.
- $F \subseteq Q$ — множество конечных (допускающих) состояний.

Применительно к тестированию Cubit:

- Q — множество возможных состояний (классов State) Cubit.
- Σ — множество публичных методов Cubit, которые могут изменять его состояние. Каждый метод можно рассматривать как событие.
- $\delta(q, m)$ — состояние, в которое переходит Cubit из состояния q после вызова метода m . Если метод не меняет состояние, то $\delta(q, m) = q$.
- q_0 — начальное состояние Cubit, задаваемое в его конструкторе.
- F — может быть всем множеством Q , если нет специфических «конечных» состояний с точки зрения тестирования.

Поведение Cubit моделируется как последовательность переходов в автомате, инициируемых вызовами его методов. Тестирование сводится к проверке того, что реальное поведение Cubit соответствует поведению, предсказанному моделью-автоматом.

1.3.2 Тестовые множества и полнота тестирования

Основная проблема при тестировании на основе автоматов — потенциально бесконечное число возможных последовательностей вызовов (путей в автомате). Необходимо выбрать конечное подмножество путей, которое было бы достаточным для проверки корректности. Эта проблема решается использованием концепции тестовых множеств, разработанной в теории формальных языков и автоматов [AlbertCulikKarhumaki1982, JarominekKarhumakiRytterTCS].

Определение 2 (Тестовое множество, адапт. из [Plandowski1993]). *Множество T последовательностей входных символов (путей в автомате) является тестовым множеством для языка L (множества всех допустимых путей автомата), если $T \subseteq L$ и для любых двух «интерпретаций» f, g (в нашем случае, реальная реализация Cubit и его модель-автомат), если f и g совпадают на всех последовательностях из T , то они совпадают и на всем языке L .*

Идея состоит в том, чтобы найти такое конечное (и желательно небольшое) тестовое множество T , проверка на котором гарантировала бы корректность всей системы. Работа [Plandowski1993] и связанные с ней исследования [Karhumaki1992, Karhumaki1993] посвящены поиску таких тестовых множеств для различных классов формальных языков, в частности, для контекстно-свободных языков (КС-языков). Хотя поведение Cubit моделируется регулярным языком (языком конечного автомата), а не КС-языком, идеи и методы из этих работ могут быть полезны.

В [Plandowski1993] доказывается существование тестовых множеств полиномиального размера для КС-языков (Теорема 6). Это достигается путем сведения задачи к тестированию на специальных линейных КС-грамматиках и языках L_k , для которых строится тестовое множество T_k .

Лемма 1 (Адаптация Леммы 5 из [Plandowski1993]). Для определенного класса автоматов (аналогичных L_4 в работе) существует конечное, небольшое тестовое множество (T_4).

Это показывает принципиальную возможность нахождения компактных тестовых множеств. Хотя прямая адаптация конструкций L_k и T_k к автоматам состояний Cubit может быть нетривиальной [Jaffar1990, KoscielskiPacholski1990], общая идея поиска конечного репрезентативного набора путей остается центральной.

1.3.3 Цели тестового покрытия для автоматов

При генерации тестов на основе модели конечного автомата основной задачей является достижение достаточного структурного покрытия графа состояний. Бессмысленно пытаться покрыть все возможные (потенциально бесконечные) пути, поэтому используются стратегии, заключающиеся в проверке всех ключевых элементов структуры автомата.

Наиболее важным и практически достижимым критерием является покрытие переходов (Transition/Edge Coverage), также известное как реберное покрытие. Этот критерий необходим для гарантии того, что каждый возможный переход (q_i, m, q_j) в автомате будет выполнен хотя бы один раз в рамках сгенерированных тестовых сценариев. Это позволяет проверить реакцию компонента (Cubit) на каждый релевантный метод m в каждом состоянии q_i , из которого этот метод может быть вызван и привести к изменению состояния. Систематический метод генерации путей, основанный на построении остова дерева BFS и добавлении путей для ребер вне дерева (описанный в разделе 4.2), нацелен именно на подобное достижение полного покрытия переходов [Karhumaki1992].

Так же имеется более слабый критерий - покрытие состояний (State Coverage), требующее лишь посещения каждого достижимого состояния автомата хотя бы один раз. Хотя это необходимое условие, оно не гарантирует проверку всех переходов между этими состояниями. Как правило, полное покрытие переходов автоматически влечет за собой и полное покрытие всех достижимых состояний.

Другие подходы, такие как покрытие всех путей до определенной длины k , являются менее надежными и эффективными, поскольку они могут генерировать большое количество избыточных тестов, повторяющих одни и те же переходы,

более того, при наличии циклов в автомате (что типично для моделей состояний) невозможно покрыть все пути даже ограниченной длины.

Таким образом, в рамках данной работы (конкретно генерация тестов Cubit) достижение полного покрытия переходов графа состояний будет достигаться алгоритмом, основанным на остоном дереве [Lothaire1983]. Это обеспечивает практический способ построения тестового набора, который систематически проверяет основную логику переходов состояний компонента.

1.4 Генерация тестов для Cubit с использованием автоматов

1.4.1 Построение автомата состояний

Процесс построения автомата $M = (Q, \Sigma, \delta, q_0, F)$ для заданного Cubit включает следующие шаги:

1. Определение множества состояний Q : Множество Q формируется из всех классов, наследующих базовый класс состояния (State) данного Cubit. Каждое уникальное состояние Cubit становится состоянием автомата.
2. Определение начального состояния q_0 : Начальное состояние q_0 определяется состоянием, которое устанавливается в конструкторе Cubit.
3. Определение алфавита Σ : Алфавит Σ состоит из всех публичных методов Cubit, которые потенциально могут изменить его состояние (т.е. вызывают `emit`). Методы, которые только возвращают данные без изменения состояния, не включаются в алфавит переходов.
4. Построение функции переходов δ : Это наиболее сложный этап, требующий анализа кода методов Cubit. Для каждого состояния $q \in Q$ и каждого метода $m \in \Sigma$: — Анализируется код метода m . — Определяется, какое состояние $q' \in Q$ будет установлено (через `emit`) при вызове метода m , когда Cubit находится в состоянии q . — Если состояние изменяется на q' , то устанавливается $\delta(q, m) = q'$. — Если метод m в состоянии q не вызывает `emit` или вызывает `emit(q)`, то $\delta(q, m) = q$. — Анализ может быть усложнен условными операторами внутри методов. В простейшем случае можно рассматривать только безусловные вызовы `emit` или вызовы в наиболее вероятных ветках исполнения. Для более точного моделирования могут потребоваться более сложные методы анализа потока управления.
5. Определе-

ние конечных состояний F : Обычно все состояния считаются допустимыми, т.е. $F = Q$.

Результатом является граф состояний, представляющий модель поведения Cubit. Пример такого графа для гипотетического LoginCubit показан в Приложении Б (Рисунок ??).

1.4.2 Генерация тестовых путей

После построения автомата $M = (Q, \Sigma, \delta, q_0, F)$ необходимо сгенерировать набор тестовых путей, который будет служить основой для тестовых сценариев. Цель — получить конечное, но репрезентативное множество путей, покрывающее ключевые аспекты поведения компонента, моделируемого автоматом.

Имеется эффективный подход, описанный в [Karhumaki1992], он заключается в построении остоного дерева графа автомата с использованием обхода в ширину (BFS), начиная с начального состояния q_0 . Затем генерируется набор путей, включающий:

1. Пути в остоном дереве: Для каждого узла $q \in Q$, достижимого из q_0 , генерируется уникальный путь от q_0 до q , идущий исключительно по ребрам остоного дерева BFS. Эти пути покрывают все достижимые состояния и ребра дерева.
2. Пути с ребрами вне дерева: Для каждого ребра (q_i, m, q_j) в автомате, которое не вошло в остоное дерево BFS (т.е., ребра, ведущие к уже посещенным на том же или более раннем уровне узлам — обратные, прямые или перекрестные ребра относительно дерева BFS), генерируется путь. Этот путь состоит из: пути в остоном дереве от q_0 до q_i , самого ребра (q_i, m, q_j) и (опционально) пути в остоном дереве от q_j до какого-либо другого узла или обратно к q_0 , если это формирует осмысленный сценарий или цикл. Эти пути необходимы для проверки всех переходов автомата, включая те, что создают циклы или альтернативные маршруты между состояниями.

Теоретически, набор путей, построенный таким образом, формирует основу для тестового множества, достаточного для проверки эквивалентности автоматов в определенных условиях [Karhumaki1992].

На практике, для генерации тестов для Cubit, мы можем адаптировать этот подход:

1. Строим остовное дерево BFS графа состояний Cubit из начального состояния.
2. Генерируем пути из дерева BFS до каждого достижимого состояния.
3. Для каждого перехода (q_i, m, q_j) , не вошедшего в дерево, генерируем путь: (путь BFS до q_i) $\rightarrow$ (переход m в q_j).
4. Из полученного набора путей выбираем до 10 наиболее разнообразных или покрывающих максимальное число уникальных переходов, возможно, ограничивая их максимальную длину для практичности.
5. Далее, транслируя имеющиеся пути в последовательный набор методов, формируем сам тест, получая из граничных узлов пути необходимое для входа и ожидаемое состояние.

Этот метод обеспечивает более структурированный и теоретически обоснованный подход к выбору тестовых путей по сравнению с простым BFS/DFS или случайным обходом, стремясь покрыть все состояния и переходы автомата.

Пример: Генерация путей для CounterCubit

Рассмотрим простой CounterCubit, который управляет одним целочисленным состоянием (счетчиком) и имеет два метода: `increment()` и `decrement()`.

1. Автомат состояний: В данном случае автомат имеет только одно формальное состояние $q_0 = \text{CounterState}(\text{value})$, так как тип состояния не меняется. Начальное состояние, допустим, $q_0 = \text{CounterState}(0)$. Алфавит $\Sigma = \{\text{increment}, \text{decrement}\}$. Функция переходов δ :

— $\delta(\text{CounterState}(v), \text{increment}) = \text{CounterState}(v+1)$

— $\delta(\text{CounterState}(v), \text{decrement}) = \text{CounterState}(v-1)$

Графически это можно представить как один узел с двумя петлями (см. Рисунок ??). 2. Построение остовного дерева BFS: Начиная с $q_0 = \text{CounterState}(0)$, обход в ширину не добавит новых узлов, так как все переходы являются петлями. Остовное дерево BFS состоит только из корневого узла q_0 . 3. Пути в остовном дереве (In-tree): Единственный путь — это пустой путь, начинающийся и заканчивающийся в q_0 . Этот путь соответствует инициализации Cubit.

— $P_{tree_1} : q_0$ (пустой путь)

4. Ребра вне дерева (Out-tree): Оба перехода являются ребрами вне дерева (петлями):

— $e_1 = (q_0, \text{increment}, q_0)$

— $e_2 = (q_0, \text{decrement}, q_0)$

5. Генерация путей по ребрам вне дерева: Согласно алгоритму, для каждого ребра (q_i, m, q_j) вне дерева строится путь: (путь BFS до q_i) $\rightarrow$ (переход m в q_j).

— Для e_1 : (путь BFS до q_0) $\rightarrow$ (increment в q_0). Путь BFS до q_0 - пустой. Получаем путь: $P_{out_1} : q_0 \xrightarrow{\text{increment}} q_0$.

— Для e_2 : (путь BFS до q_0) $\rightarrow$ (decrement в q_0). Путь BFS до q_0 - пустой. Получаем путь: $P_{out_2} : q_0 \xrightarrow{\text{decrement}} q_0$.

6. Итоговый набор базовых путей: Объединяя пути из дерева и пути по ребрам вне дерева, получаем базовый набор:

— $P_1 : q_0$ (Инициализация)

— $P_2 : q_0 \xrightarrow{\text{increment}} q_0$ (Инкремент)

— $P_3 : q_0 \xrightarrow{\text{decrement}} q_0$ (Декремент)

Этот пример иллюстрирует применение алгоритма на основе остовного дерева для систематической генерации тестовых путей, покрывающих все основные переходы состояния, даже в простом случае с одним состоянием и петлями.

1.4.3 Трансляция путей в тестовый код

Каждый сгенерированный путь $P = (q_0 \xrightarrow{m_1} q_1 \xrightarrow{m_2} q_2 \dots \xrightarrow{m_k} q_k)$ транслируется в отдельный тестовый случай (например, функцию `test(...)` в Dart).

Код теста выполняет следующие шаги: 1. Создает экземпляр тестируемого Cubit. Начальное состояние должно быть q_0 . 2. Использует подходящий фреймворк для тестирования потоков состояний (например, `bloc_test`). 3. Последовательно вызывает методы $m_1, m_2, \dots, m_k$. Для методов с аргументами генерируются значения по умолчанию или заглушки. 4. После каждого вызова m_i проверяет, что Cubit перешел в ожидаемое состояние q_i . Фреймворк `bloc_test` позволяет элегантно описывать ожидаемую последовательность состояний (`expect: () => <Matcher>[q_1, q_2, ..., q_k]`). 5. При необходимости мокирует зависимости Cubit.

Пример сгенерированного теста для одного пути показан в Приложении А (Листинг 5).

1.5 Анализ подхода и анализ релевантности

1.5.1 Преимущества

- Автоматизация: Снижение ручного труда при написании модульных тестов.
- Стандартизация: Генерация тестов по единым правилам и шаблонам.
- Покрытие: Автоматическое создание тестов для различных сценариев, включая граничные случаи (для статической генерации) и последовательности переходов состояний (для динамической генерации).
- Теоретическая основа: Использование конечных автоматов и идей тестовых множеств для Cubit обеспечивает более систематический подход к тестированию компонентов с состоянием по сравнению с ручным написанием.

1.5.2 Ограничения и сложности

— Статический анализ: Возможности статического анализа ограничены. Сложно автоматически определить корректные ожидаемые значения для методов со сложной логикой или точно настроить моки для всех зависимостей. Сгенерированные статические тесты часто требуют доработки. — Построение автомата: Автоматическое и точное построение функции переходов δ для Cubit может быть сложным из-за условной логики, асинхронных операций и внешних зависимостей внутри методов. Модель может быть упрощенной. — Выбор путей: Выбор ограниченного числа путей (до 10) не гарантирует полного покрытия всех возможных сценариев, особенно для сложных автоматов с большим количеством циклов. Критерии покрытия (состояния, переходы) помогают, но не дают абсолютной гарантии. — Тестовые данные: Генерация осмысленных тестовых данных для аргументов методов (особенно сложных объектов) остается проблемой как для статической, так и для динамической генерации.

1.5.3 Анализ релевантности

Для оценки практической значимости и потенциальной экономии времени при использовании предлагаемой системы автоматической генерации тестов

был проведен анализ характеристик реальных компонентов управления состоянием (Cubit) из существующих проектов. В рамках анализа было рассмотрено 15 различных Cubit'ов разной степени сложности.

Результаты анализа показали следующие средние характеристики:

- Количество различных состояний (классов State) на один Cubit: 5-6.
- Количество методов, изменяющих состояние (содержащих вызов emit), т.е. количество типов переходов в автомате: 4.

Эти данные показывают, что даже для относительно простых Cubit'ов количество возможных путей в автомате состояний может быть значительным, что усложняет полное ручное тестирование всех сценариев.

Также была проведена оценка временных затрат на ручное написание модульных тестов для Cubit'ов:

- Среднее время написания тестов опытным разработчиком: 3 часа. Стоит отметить, что в стандартные оценки трудоемкости задач по разработке часто закладывается больше времени на тестирование (до 6 часов), что указывает на потенциальную недооценку или нехватку времени на полное тестирование при ручном подходе.
- Среднее время написания тестов новым или менее опытным разработчиком: 4 часа.

Предполагаемое использование плагина для автоматической генерации тестов, согласно оценкам, может привести к следующим улучшениям и изменениям опыта разработки: — Сокращение времени разработки: Ожидается экономия примерно 2 часов на написание тестов для одного Cubit. Это достигается за счет автоматической генерации каркаса тестов и основных сценариев на основе автомата состояний. Разработчику остается только доработать тесты, настроить моки и добавить специфические проверки. — Увеличение тестового покрытия: Автоматическая генерация на основе обхода автомата позволяет выявить и покрыть сценарии (пути), которые могли бы быть упущены при ручном написании. Ожидается, что использование плагина позволит добавить в среднем 3-4 дополнительных тестовых сценария по сравнению со стандартным набором, создаваемым вручную, что напрямую повышает полноту и надежность тестирования.

Таким образом, внедрение системы автоматической генерации тестов является релевантным и экономически целесообразным шагом, позволяющим не

только ускорить процесс разработки, но и повысить качество итогового продукта за счет более полного тестового покрытия.

1.6 Обзор предметной области

1.6.1 Используемые технологии

Разработка плагина базируется на современном технологическом стеке, обеспечивающем эффективную интеграцию с экосистемой Flutter-разработки.

Visual Studio Code Extension API

Visual Studio Code был выбран в качестве целевой платформы для разработки плагина по следующим причинам: — Широкое распространение среди Flutter-разработчиков: VS Code является одной из наиболее популярных IDE для разработки на Dart/Flutter благодаря официальной поддержке от команды Flutter. — Развитая экосистема расширений: VS Code предоставляет мощный API для создания расширений, включающий возможности работы с файловой системой, редактором кода, пользовательским интерфейсом и интеграцией с внешними инструментами. — Кроссплатформенность: расширения VS Code работают на Windows, macOS и Linux, что обеспечивает широкую доступность плагина. — Интеграция с инструментами разработки: встроенная поддержка терминала, системы контроля версий и отладчика упрощает процесс разработки и тестирования.

text

TypeScript выбран в качестве основного языка разработки плагина: — Статическая типизация: TypeScript обеспечивает проверку типов на этапе компиляции, что значительно снижает количество ошибок времени выполнения и повышает надежность кода. — Совместимость с JavaScript: возможность использования существующих JavaScript-библиотек и постепенного перехода к типизированному коду. — Развитые инструменты разработки: автодополнение, рефакторинг и навигация по коду в современных IDE значительно превосходят возможности для обычного JavaScript. — Официальная поддержка VS Code API: все типы и интерфейсы VS Code Extension API предоставляются в TypeScript, что упрощает разработку и снижает вероятность ошибок. — Объектно-ориентированное программирование: поддержка классов, интерфейсов и модулей обеспечивает лучшую структуризацию кода сложных проектов.

Dart и Flutter Framework

Dart и Flutter представляют целевую экосистему для генерируемых тестов: — Dart: современный объектно-ориентированный язык программирования, разработанный Google. Обеспечивает высокую производительность, поддержку асинхронного программирования и развитую систему типов. — Flutter: кроссплатформенный фреймворк для разработки мобильных, веб и desktop приложений. Использует декларативный подход к построению пользовательского интерфейса и предоставляет богатый набор виджетов. — Архитектурные паттерны: Flutter активно использует паттерны управления состоянием, такие как BLoC (Business Logic Component) и его упрощенная версия Cubit, которые являются основными объектами для генерации тестов в разработанном плагине.

Библиотека `flutter_test`

Библиотека `flutter_test` является основным инструментом для модульного тестирования во Flutter: — Встроенная поддержка: входит в стандартный SDK Flutter, не требует дополнительной установки. — Comprehensive API: предоставляет функции `test()`, `group()`, `setUp()`, `tearDown()` для структурирования тестов. — Матчеры и assertions: богатый набор функций `expect()` с различными матчерами для проверки результатов тестирования. — Поддержка асинхронного тестирования: встроенные механизмы для тестирования `Future` и `Stream` объектов.

Библиотека `bloc_test`

Специализированная библиотека `bloc_test` предназначена для тестирования BLoC и Cubit компонентов: — Декларативный синтаксис: функция `blocTest()` предоставляет структурированный подход к описанию тестовых сценариев с блоками `build`, `act`, `expect`, `verify`. — Тестирование последовательностей состояний: автоматическая проверка порядка эмиссии состояний в ответ на события или вызовы методов. — Интеграция с mock-библиотеками: `seamless` работа с `mockito` для создания mock-объектов зависимостей. — Поддержка сложных сценариев: возможность тестирования условных переходов, обработки ошибок и асинхронных операций. — Отладочные возможности: детальный вывод информации о фактических и ожидаемых состояниях при сбое тестов.

Выбранный технологический стек обеспечивает создание надежного, поддерживаемого и легко расширяемого инструмента, полностью интегрированного в экосистему Flutter-разработки.

1.6.2 Тестирование в разработке на Flutter

Flutter, как современный фреймворк для создания кроссплатформенных приложений, предоставляет развитые инструменты для тестирования. Основными видами тестов являются модульные тесты, виджет-тесты и интеграционные тесты. Модульные тесты проверяют логику отдельных функций, методов или классов и выполняются быстрее всего. Для управления состоянием в Flutter-приложениях часто используется паттерн BLoC, в частности его упрощенная версия - Cubit. Тестирование Cubit включает проверку корректности переходов между состояниями в ответ на вызовы его методов. Пакет значительно упрощает написание таких тестов.

1.6.3 Проблемы ручного тестирования и необходимость автоматизации

Несмотря на наличие инструментов, ручное написание модульных тестов, особенно для Cubit, требует значительных временных затрат. Разработчику необходимо:

- Создать тестовый файл с корректной структурой и импортами.
- Для каждого значимого сценария описать последовательность действий (вызовов методов) и ожидаемых состояний.
- Учесть различные комбинации входных данных и начальных состояний.
- Поддерживать тесты в актуальном состоянии при изменении логики компонента.

Этот процесс не только трудоемок, но и подвержен человеческому фактору: возможен пропуск важных тестовых случаев или допущение ошибок в самих тестах. Автоматизация генерации базовых тестовых сценариев может существенно снизить эту нагрузку.

1.6.4 Постановка задачи разработки плагина

Задача состоит в создании расширения (плагина) для VS Code, которое позволит разработчикам генерировать заготовки модульных тестов для Dart-кода. Плагин должен: 1. Активироваться из контекстного меню редактора кода. 2. Ана-

лизировать выбранный класс Cubit/BLoC, DTO или метод. 3. Для Cubit/BLoC:

- Идентифицировать его состояния (классы State).

- Находить публичные методы и ивенты со связанными с ними приватными методами, изменяющие состояние (вызывающие emit).

- Генерировать тестовые случаи с использованием bloc_test, покрывающие различные последовательности вызовов методов и переходов состояний.

4. Для DTO:

- Генерировать тестовые случаи, включая объявление Json и Entity, покрывающие такие случаи как: непришедшее поле, поле неправильно типа и лишние поля.

5. Для методов:

- Генерировать шаблон теста, включая инициализацию класса (если метод не статический) и вызов метода с заглушками для аргументов и ожидаемого результата.

6. Корректно формировать пути для тестовых файлов и необходимые импорты.

7. Предоставлять сгенерированный код в виде, легко расширяемом и дорабатываемом пользователем.

2 Конструкторская часть

2.1 Проектирование плагина

2.1.1 Архитектура плагина

Плагин разрабатывается как расширение для Visual Studio Code с использованием языка TypeScript. VS Code предоставляет API для взаимодействия с редактором, файловой системой и пользовательским интерфейсом.

Основные компоненты архитектуры плагина: — Компонент инициализации (`extension.ts`): регистрирует команды плагина в VS Code и обрабатывает активацию расширения. — Обработчик команд: активируется при вызове пользователем функции генерации тестов. Определяет контекст (выбранный файл, позиция курсора) для анализа. — Лексический анализатор (`dartLexer.ts`): современный токенизатор Dart-кода, поддерживающий полную грамматику языка, включая сложные операторы, типы данных, строки с escape-последовательностями и импорты. — Парсер Dart-кода: отвечает за анализ содержимого файла `.dart` для извлечения информации о классах, методах, состояниях Cubit/BLoC, импортах и структурных элементах. — Генератор тестов для Cubit/BLoC (`cubitGenerator.ts`): на основе информации от парсера строит тестовые сценарии для Cubit/BLoC с поддержкой автоматов состояний и визуализации. — Генератор тестов для DTO (`dtoGenerator.ts`): на основе информации от парсера строит тестовые сценарии парсинга JSON с автоматическим поиском связанных Entity. — Генератор тестов для методов (`methodGenerator.ts`): создает шаблоны тестов для отдельных методов. — Генератор веб-интерфейса (`webviewGenerator.ts`): создает интерактивные HTML-панели с визуализацией автоматов состояний, тестовых путей и детальной аналитикой в формате Graphviz. — Модуль утилит (`utils.ts`): централизованные функции преобразования строк, константы состояний, регулярные выражения и общие вспомогательные функции. — Модуль управления файлами: создает или обновляет тестовые файлы в правильной директории с автоматическим форматированием кода.

Система автоматической генерации тестов реализована в виде модульной архитектуры, состоящей из нескольких взаимосвязанных компонентов. Каждый компонент отвечает за определенный аспект анализа кода и генерации тестов,

обеспечивая высокую степень разделения ответственности и возможность независимого тестирования модулей.

2.1.2 Основные интерфейсы и структуры данных

Архитектура системы базируется на наборе ключевых интерфейсов, определяющих структуру данных для представления различных аспектов анализируемого кода.

StateInfo - Информация о состояниях

```
1 interface StateInfo {  
2     name: string;           // Название состояния  
3     properties?: string[];  // Свойства состояния  
4     isInitial?: boolean;    // Является ли начальным состоянием  
5     isError?: boolean;      // Является ли состоянием ошибки  
6     isLoading?: boolean;    // Является ли состоянием загрузки  
7     isSuccess?: boolean;    // Является ли успешным состоянием  
8 }
```

Интерфейс StateInfo представляет метainформацию о состояниях Bloc/Cubit компонентов. Он включает классификацию состояний по типам (начальное, ошибка, загрузка, успех), что позволяет генератору тестов создавать более осмысленные тестовые сценарии.

EnhancedStateInfo - Расширенная информация о состояниях

```
1 interface EnhancedStateInfo extends StateInfo {  
2     isReachable?: boolean;    // Достижимость состояния  
3     reachabilityPaths?: string[]; // Пути достижения состояния  
4 }
```

Расширенная версия интерфейса состояний, добавляющая информацию о достижимости состояний в рамках автомата состояний и возможных путях их достижения.

DependencyInfo - Информация о зависимостях

```
1 interface DependencyInfo {  
2     name: string;           // Имя переменной зависимости  
3     type: string;           // Тип зависимости  
4 }
```

Интерфейс описывает зависимости Cubit/BLoC компонентов, такие как репозитории и сервисы, которые внедряются через конструктор и требуют создания mock-объектов в тестах.

EventInfo - Информация о событиях

```
1 interface EventInfo {  
2     name: string;           // Название события  
3     handler: string;        // Название метода-обработчика  
4 }
```

Интерфейс описывает события в Bloc-архитектуре, устанавливая связь между событиями и их обработчиками.

ParsedMethod - Разобранный метод

```
1 interface ParsedMethod {  
2     name: string;           // Название метода/события  
3     type: 'event' | 'method'; // Тип: событие (для Bloc) или метод  
                                (для Cubit)  
4     allowedFromStates: string[]; // Состояния, из которых можно вызвать  
5     emitStates: string[];       // Состояния, которые эмитируются  
6     guardConditions: string[];  // Guard условия (запрещенные состояния)  
7 }
```

Базовая структура для представления методов и событий с основной информацией о переходах состояний.

EnhancedMethodInfo - Расширенная информация о методах

```
1 interface EnhancedMethodInfo {  
2     name: string;           // Название метода/события  
3     type: 'event' | 'method'; // Тип: событие (для Bloc) или метод  
                                (для Cubit)  
4     event?: string;         // Само событие (для блоков)  
5     handler?: string;       // Обработчик (для блоков)  
6     parameters: string[];   // Параметры  
7     transitions: StateTransition[]; // Детальные переходы между  
                                состояниями  
8     reachableStates: string[]; // Все достижимые состояния из этого  
                                метода  
9     finalStates: string[];   // Конечные состояния метода  
10    transientStates: string[]; // Сквозные состояния метода  
11    repositoryCalls: string[]; // Вызовы репозитория
```

```

12     storageCalls: string[];        // Вызовы хранилища
13     privateMethodCalls: string[]; // Вызовы частных методов
14     guardConditions: GuardCondition[]; // Guard условия
15     allowedStates: string[];       // Разрешенные состояния для вызова
16     isArrowFunction?: boolean;     // Является ли стрелочной функцией
17 }

```

Детальная структура метода/события, включающая полную информацию о переходах состояний, вызовах зависимостей и ограничениях.

StateTransition - Переход между состояниями

```

1 interface StateTransition {
2     fromState: string;           // Начальное состояние
3     toState: string;            // Конечное состояние
4     transientStates: string[];   // Сквозные состояния между началом и
        концом
5     repositoryCalls: string[];   // Вызовы методов репозитория
6     storageCalls: string[];      // Вызовы методов хранилища
7     privateMethodCalls: string[]; // Вызовы частных методов
8     isReachable: boolean;        // Достижимость перехода
9     conditionGuards: string[];   // Условия/ограничения перехода
10 }

```

Детальное описание перехода между состояниями, включая информацию о промежуточных состояниях и вызовах внешних зависимостей.

GuardCondition - Guard условие

```

1 interface GuardCondition {
2     blockedState: string;        // Заблокированное состояние
3     action: string;             // Действие (обычно "return")
4 }

```

Описывает ограничения на выполнение методов в определенных состояниях (например, ранние возвраты).

DartMethodContext - Контекст метода Dart

```

1 interface DartMethodContext {
2     blockType: 'try' | 'catch' | 'if' | 'else' | 'switch' | 'case' |
        'default' | 'root';
3     startIndex: number;
4     endIndex: number;

```

```

5     nestingLevel: number;
6     parentContext?: DartMethodContext;
7 }

```

Представляет контекст выполнения внутри методов Dart для анализа условных блоков и обработки исключений.

EmitInfo - Информация об emit вызове

```

1 interface EmitInfo {
2     state: string;           // Нормализованное название состояния
3     rawStatement: string;    // Оригинальный emit statement
4     context: DartMethodContext | null;
5     isTransient: boolean;
6     isFinal: boolean;
7     reason: string;
8 }

```

Детальная информация о вызовах emit(), включая контекст и классификацию по типу (транзитное или финальное состояние).

FSM - Автомат конечных состояний

```

1 interface FSM {
2     states: string[];        // Все состояния в автомате
3     transitions: FSMTransition[]; // Все возможные переходы
4     initialState: string;    // Начальное состояние
5 }
6
7 interface FSMTransition {
8     from: string;            // Исходное состояние
9     to: string;              // Целевое состояние
10    method: string;           // Метод/событие, вызывающее переход
11    condition?: string;       // Условие перехода
12 }

```

Формальная модель конечного автомата, построенного на основе анализа Bloc/Cubit компонента.

TestPath - Тестовый путь

```

1 interface TestPath {
2     states: string[];        // Последовательность состояний в пути
3     methods: string[];       // Последовательность методов/событий

```

```

4     conditions: (string | undefined)[]; // Условия для каждого перехода
5 }

```

Представляет тестовый сценарий как последовательность состояний и методов с условиями переходов.

ExecutionBranch - Ветка выполнения

```

1 interface ExecutionBranch {
2     branchId: string;           // Уникальный идентификатор ветки
3     states: string[];          // Последовательность состояний
4     repositoryCalls: string[]; // Вызовы методов репозитория
5     storageCalls: string[];     // Вызовы методов хранилища
6     privateMethodCalls: string[]; // Вызовы приватных методов
7     isSuccessPath: boolean;     // Путь успеха
8     isErrorPath: boolean;       // Путь с ошибкой
9     isFinalState: boolean;      // Конечное состояние
10 }

```

Описывает отдельную ветку выполнения в методе, включая информацию о вызовах внешних зависимостей, что необходимо для генерации правильных моков-настроек в тестах.

Token и TokenType - Лексический анализ

```

1 interface Token {
2     type: TokenType;
3     value: string;
4     position: number;
5     line: number;
6     column: number;
7 }
8
9 enum TokenType {
10     CLASS, IMPORT, FUNCTION, ASYNC, AWAIT, IF, ELSE, TRY, CATCH,
11     SWITCH, CASE, DEFAULT, RETURN, EMIT, IDENTIFIER, TYPE,
12     STRING, NUMBER, BOOLEAN, CHAR, FLOAT, DOUBLE,
13     ASSIGN, EQUALS, NOT_EQUALS, LT_EQUALS, GT_EQUALS,
14     LPAREN, RPAREN, LBRACE, RBRACE, LBRACKET, RBRACKET,
15     WHITESPACE, NEWLINE, COMMENT, EOF, UNKNOWN
16 }

```

Базовые структуры лексического анализатора для токенизации Dart-кода с поддержкой полной грамматики языка.

DataItem - Информация о полях DTO

```
1 interface DataItem {
2     dartName: string;           // Имя переменной в Dart-нотации
3     jsonKey: string;           // Имя переменной в JSON-нотации
4     type: string;              // Тип переменной
5 }
```

Специализированный интерфейс для генерации тестов DTO, описывающий маппинг между JSON-ключами и полями Dart-класса.

ASTNode - Узлы абстрактного синтаксического дерева

```
1 abstract class ASTNode {
2     public type: string;
3     constructor(type: string) { this.type = type; }
4 }
5
6 class ClassNode extends ASTNode {
7     constructor(
8         public name: string,
9         public extendsClass?: string,
10        public body: ASTNode[] = []
11    ) { super('class'); }
12 }
13
14 class MethodNode extends ASTNode {
15     constructor(
16         public name: string,
17         public returnType: string,
18         public isAsync: boolean = false,
19         public parameters: string[] = [],
20         public body: ASTNode[] = []
21    ) { super('method'); }
22 }
```

Базовые классы для представления элементов кода в виде абстрактного синтаксического дерева, используемые в перспективном AST-парсере.

2.1.3 Основные функциональные компоненты

Анализатор кода

Компонент анализа кода включает функции для статического анализа Dart-кода и извлечения структурной информации:

- `parseInlineStates()` — парсинг состояний, определенных в том же файле, что и Bloc/Cubit.
- `parseStates()` — анализ внешних файлов состояний.
- `parseCubitOrBloc()` — извлечение информации о методах, событиях и зависимостях.
- `parseMethodTransitions()` — анализ переходов состояний внутри методов.
- `parseRepositoryCalls()` и `parseStorageCalls()` — выявление вызовов внешних зависимостей.

Построитель автоматов

Модуль построения конечных автоматов преобразует результаты анализа кода в формальную модель:

- `buildFSM()` — основная функция построения автомата состояний из проанализированных компонентов.
- `analyzeStates()` — классификация состояний на транзитные и финальные.
- `analyzeEmitContext()` — анализ контекста вызовов `emit` для определения условий переходов.

Генератор тестовых путей

Компонент для генерации тестовых путей на основе теории графов:

- `generateTestPaths()` — основная функция генерации путей с ограничением по количеству.
- `buildBFSSpanningTree()` — построение остовного дерева обходом в ширину.
- `generateInTreePaths()` — генерация путей в остовном дереве.
- `generateOutTreePaths()` — генерация путей для рёбер вне остовного дерева.
- `removeDuplicatePaths()` — удаление дублирующихся путей.

Генератор тестового кода

Модуль трансляции тестовых путей в исполняемый код тестов:

- `generateBlocTests()` — основная функция генерации тестов для Bloc/Cubit.
- `generateMinimalRequiredTests()` — генерация обязательных тестов для всех методов.
- `generateSingleMethodTest()` — генерация теста для отдельного метода.
- `generateMockSetup()` — создание настроек mock-объектов.
- `generateStateInstance()` — генерация экземпляров состояний.

Интеграция с VS Code

Компонент интеграции обеспечивает взаимодействие с API редактора:

- `generateTest()` — главная экспортируемая функция плагина.
- `generateFeatureTestFile()` — создание структуры тестов на уровне feature.
- `updateMainTestFile()` — обновление главного файла тестов проекта.
- `applyAllCodeActions()` — применение автоматического форматирования.
- `runTestsForFile()` — запуск сгенерированных тестов.

Лексический анализатор Dart

Система лексического анализа представляет собой современный токенизатор, реализующий полную поддержку грамматики Dart:

- Поддержка операторов: Полная поддержка всех операторов сравнения (`<=`, `>=`, `!=`, `==`), арифметических операторов и операторов присваивания.
- Типы данных: Распознавание всех примитивных типов Dart (`int`, `double`, `String`, `bool`, `char`, `float`) и их вариантов.
- Обработка строк: Корректная обработка строковых литералов с escape-последовательностями (`\n`, `\t`, `\"`) и различными типами кавычек.
- Числовые литералы: Поддержка положительных, отрицательных чисел, чисел с плавающей точкой, суффиксов типов (`f`, `d`) и различных форматов записи.
- Ключевые слова: Распознавание всех ключевых слов Dart, включая модификаторы доступа, управляющие конструкции и объявления типов.

- Импорты: Специальная обработка `import` выражений для корректного построения зависимостей между модулями.
- Комментарии: Поддержка как однострочных (`//`), так и многострочных (`/* */`) комментариев с возможностью их игнорирования или сохранения.

Лексер построен на основе конечного автомата, обеспечивающего высокую производительность и точность анализа даже для сложных конструкций Dart-кода.

2.1.4 Порядок выполнения

Общий порядок выполнения системы включает следующие этапы:

1. Инициализация: Пользователь активирует команду генерации тестов в VS Code.
2. Анализ кода: Система анализирует выбранный файл и связанные компоненты.
3. Построение модели: На основе анализа строится автомат состояний (для Bloc/Cubit) или извлекается структурная информация (для DTO/методов).
4. Генерация путей: Для автоматов генерируются тестовые пути с использованием алгоритма остовного дерева.
5. Создание тестов: Пути транслируются в исполняемый код тестов с автоматической настройкой mock-объектов.
6. Интеграция: Созданные тесты интегрируются в структуру проекта с обновлением главных тестовых файлов.

2.1.5 Взаимодействие с пользователем

Плагин интегрируется в VS Code через контекстное меню, появляющееся при вызове контекстного меню (`Ctrl+Shift+.` / `Cmd+.`) на названии класса Cubit/BLoC, DTO или метода в редакторе кода. Команда «Flutter: Generate test» инициирует процесс анализа и генерации.

Результатом является создание структуры тестовых файлов в соответствующих директориях проекта. При генерации тестов для Cubit/BLoC плагин дополнительно создает интерактивную веб-панель (webview), которая предоставляет:

- Визуализацию автомата состояний: Интерактивный граф в формате Graphviz, показывающий все состояния компонента и возможные переходы между ними.
- Отображение тестовых путей: Граф тестовых множеств с цветовой дифференциацией путей и использованием различной толщины линий для уникальной идентификации каждого пути.
- Детальный анализ методов: Сворачивающиеся секции с информацией о каждом методе/событии, включая список достижимых состояний, вызовы зависимостей и ветки выполнения.
- Информацию о ветках выполнения: Структурированное отображение всех найденных путей выполнения с указанием типа пути (успех/ошибка) и связанных вызовов.
- Копирование в буфер обмена: Возможность копирования DOT-кода графов для использования в внешних инструментах визуализации.
- Интеграция с Graphviz Online: Прямая ссылка на онлайн-сервис для редактирования и расширенной визуализации графов.

Веб-интерфейс использует современные технологии HTML5, CSS3 и JavaScript для обеспечения удобного взаимодействия и высокой информативности отображаемых данных.

2.1.6 Парсинг исходного кода

Парсинг Dart-кода осуществляется с использованием комбинации лексического анализа и целевых регулярных выражений для идентификации ключевых конструкций:

- Лексический анализ: Входной код разбивается на токены с использованием специализированного лексера, поддерживающего полную грамматику Dart, включая сложные операторы, типы данных и строковые литералы.
- Определение структуры классов: Извлечение информации о классе Cubit/BLoC, его базовом классе состояния и иерархии наследования.
- Анализ состояний: Поиск всех классов состояний, наследующих базовое состояние, с определением их свойств, конструкторов и семантических характеристик.
- Парсинг методов и событий: Для Cubit анализируются публичные методы, для BLoC - события и их обработчики, с извлечением полной сигнатуры и параметров.
- Анализ emit-вызовов: Де-

тальный разбор тел методов для обнаружения всех вызовов `emit()`, определения контекста (условные блоки, `try-catch`) и классификации состояний. — Трассировка зависимостей: Поиск и анализ вызовов репозиториев, сервисов и приватных методов для автоматической генерации mock-настроек. — Анализ импортов: С использованием лексера извлекаются все импорты файла, включая относительные пути и псевдонимы, для корректного формирования зависимостей в тестах. — Определение Guard-условий: Поиск ограничений на вызовы методов (например, ранние возвраты при определенных состояниях).

Данный подход обеспечивает высокую точность анализа и устойчивость к различным стилям написания кода, что критически важно для корректности генерируемых тестов.

3 Технологическая часть

3.1 Реализация ключевых функций

3.1.1 Генерация тестов для Cubit/BLoC

Процесс генерации тестов для Cubit/BLoC является наиболее сложным компонентом плагина и включает несколько этапов анализа и генерации.

Полный алгоритм работы с Cubit/BLoC

Этап 1: Парсинг и извлечение структурных элементов

Плагин выполняет комплексный анализ исходного кода для извлечения следующих элементов: — Идентификация классов состояний: Парсер ищет все классы, наследующие базовое состояние (State). Для каждого состояния определяются его свойства, конструкторы и семантические характеристики (является ли состояние начальным, ошибочным, состоянием загрузки или успеха). — Анализ зависимостей: Извлекаются все зависимости Cubit/BLoC (репозитории, сервисы), которые внедряются через конструктор. Это необходимо для последующего создания mock-объектов. — Парсинг методов/ивентов: Для Cubit анализируются публичные методы, для BLoC - события (Event) и соответствующие им методы-обработчики. Определяются параметры каждого метода/ивента и их типы. — Анализ тел методов/обработчиков: Детальный разбор кода методов для обнаружения вызовов `emit()`, условных конструкций, блоков `try-catch`, вызовов зависимостей.

Этап 2: Построение модели конечного автомата

На основе извлеченной информации строится формальная модель поведения: — Определение множества состояний *S*: Все классы состояний становятся узлами автомата. — Определение алфавита входных сигналов: Методы/ивенты формируют алфавит переходов. — Построение функции переходов: Для каждого метода/ивента анализируются возможные переходы между состояниями через вызовы `emit()`. — Выявление `guard`-условий: Определяются ограничения на переходы (например, метод может быть вызван только в определенных состояниях). — Классификация состояний: Состояния маркируются как транзитные (проходные) или финальные (завершающие) на основе анализа контекста их использования.

Этап 3: Построение тестовых множеств

Плагин применяет комбинацию алгоритмов для генерации полного набора тестовых путей: — BFS-обход для построения остоного дерева: Строится минимальное покрывающее дерево автомата, обеспечивающее достижимость всех состояний. — Генерация базовых путей: Для каждого состояния строится путь от начального состояния. — Выявление дополнительных рёбер: Находятся переходы, не включенные в остоное дерево. — Генерация комбинированных путей: Строятся сложные тестовые сценарии, покрывающие последовательности переходов. — Удаление дубликатов: Исключаются избыточные тестовые пути.

Этап 4: Анализ ветвей выполнения

Для каждого метода/ивента выполняется детальный анализ: — Парсинг условных конструкций: Выявляются ветки `if-else`, `switch-case`. — Анализ блоков `try-catch`: Определяются пути успешного выполнения и обработки ошибок. — Трассировка вызовов зависимостей: Отслеживаются вызовы методов репозитория и сервисов. — Построение графа потока управления: Для сложных методов строится детальная модель выполнения.

Этап 5: Трансляция в конечные тесты

Каждый тестовый путь транслируется в код теста `blocTest`: — Генерация блока `build`: Создается экземпляр `Cubit/BLoC` с `mock`-зависимостями. — Настройка `mock`-объектов: Для каждого пути настраиваются соответствующие ответы `mock`-зависимостей (успешные результаты или исключения). — Формирование блока `act`: Генерируется последовательность вызовов методов/отправки ивентов с типовыми параметрами. — Построение блока `expect`: Определяется ожидаемая последовательность состояний. — Добавление блока `verify`: Проверяется корректность вызовов `mock`-зависимостей.

3.1.2 Генерация тестов для **Data Transfer Object**

Генерация тестов для объектов передачи данных (`Data Transfer Objects`) реализована в `dtoGenerator.ts` и включает следующие этапы:

Парсинг структуры `Data Transfer Object`

1. Анализ метода `fromJson`: Плагин парсит конструктор `fromJson` для извлечения маппинга между `JSON`-ключами и полями `Dart`-класса. Определяются типы полей через анализ `as Type` конструкций.

2. Определение связанного Entity: Анализируется метод `toEntity()` для определения целевого класса-сущности, в который преобразуется DTO.
3. Построение модели данных: Создается внутреннее представление с информацией о каждом поле: имя в Dart-нотации, ключ в JSON, тип данных.

Генерация тестовых сценариев

Для каждого DTO генерируется комплект тестов, покрывающих различные сценарии парсинга JSON:

- Тест с корректными данными: Проверяется успешный парсинг JSON с правильной структурой и типами полей.
- Тест с лишними полями: Убеждается, что дополнительные поля в JSON не нарушают парсинг.
- Тесты на отсутствие обязательных полей: Для каждого поля генерируется тест, проверяющий обработку ситуации, когда поле отсутствует в JSON.
- Тесты на несоответствие типов: Для каждого поля проверяется поведение при получении значения неправильного типа.

Структура генерируемых тестов

Каждый тестовый файл содержит:

- Константу с тестовыми данными `<dto_name>Data`
- Ожидаемый объект Entity `<expected>`
- Группу тестов `group('Сценарий парсинга <DtoName>')`
- Набор тестовых случаев с использованием `expect` и проверкой исключений `throwsA(isA<TypeError>())`

3.1.3 Генерация тестов для методов/ивентов

Для обычных методов классов (не Cubit/BLoC) или top-level функций, `methodGenerator.ts` генерирует упрощенный шаблон теста:

1. Определение сигнатуры метода/ивента: Извлекается имя метода/ивента, его параметры и их типы, а также тип возвращаемого значения.
2. Создание тестового случая: Генерируется функция `test` из пакета `test`.
3. Инициализация: Если метод принадлежит классу, предусматривается место для инициализации экземпляра класса.

4. Вызов метода/ивента: Генерируется вызов метода/ивента с заглушками для аргументов.
5. Проверка результата: Добавляется expect с заглушкой для ожидаемого результата, которую пользователь должен будет заполнить.

Этот подход предоставляет разработчику каркас, который необходимо дозаполнить конкретными тестовыми данными и ожиданиями.

3.1.4 Решение проблем и доработки

В ходе разработки были решены следующие ключевые задачи:

Создание стабильной архитектуры — Модульная архитектура: Разделение функциональности на независимые модули (`dartLexer.ts`, `webviewGenerator.ts`, `utils.ts`) для улучшения поддерживаемости и тестируемости кода. — Централизация утилит: Создание модуля `utils.ts` с общими функциями преобразования строк, константами состояний и регулярными выражениями для предотвращения дублирования кода.

Лексического анализа — Расширение грамматики: Реализована поддержка всех операторов и большинства типов данных. — Обработка числовых литералов: Реализован корректный парсинг положительный, отрицательных чисел и чисел с плавающей точкой, включая суффиксы типов. — Строковый анализ: Реализована обработка строковых литералов с escape-последовательностями и различными типами кавычек. — Составные операторы: реализована поддержка сложных операторов (`=>`, `!`) и их корректная токенизация.

Технические решения — Корректное определение путей для импорта: Реализована логика для определения относительных путей к тестируемым файлам из тестовых файлов с поддержкой различных структур проектов. — Обработка сложных типов параметров: Реализован парсинг для определения типов аргументов методов/событий Cubit/BLoC для генерации корректных заглушек и mock-объектов. — Анализ emit-контекста: Детальный анализ контекста вызовов `emit()` для определения транзитных и финальных состояний.

Визуализация и пользовательский интерфейс — Построение графов состояний: Реализованы алгоритмы для визуализации автомата состояний и тестовых путей в формате Graphviz с поддержкой цветовой дифференциации. — Интерактивный webview: Создание современного веб-интерфейса с возможностью

копирования графов, сворачивающимися секциями и интеграцией с внешними сервисами. — Уникальная идентификация путей: Использование комбинации цветов и толщины линий для визуального различения тестовых путей в легенде.

Качество кода — Современная документация: Использован современный формат TypeScript JSDoc для всех функций и интерфейсов. — Типизация: Полная типизация всех функций и интерфейсов для предотвращения ошибок времени выполнения.

3.2 Инструкция пользователя

Данный раздел содержит пошаговое руководство по установке и использованию разработанного плагина для автоматической генерации тестов в среде VS Code.

Установка плагина

1. Скачайте файл плагина с расширением `.vsix` или выполните команду `npm run compile` из директории плагина и запустите проект нажатием `F5` для режима разработки.
2. В VS Code откройте панель расширений через меню `Extensions` или используйте сочетание клавиш `Ctrl+Shift+X`.
3. Нажмите на значок «...» (три точки) в верхней части панели расширений.
4. Выберите пункт «Install from VSIX...».
5. Укажите путь к скачанному файлу плагина и подтвердите установку.

Подготовка проекта

Для корректной работы плагина убедитесь, что ваш Flutter-проект имеет следующую структуру директорий:

```
lib/  
— features/  
    — [название_фичи]/  
        * bloc/ (или cubit/  
            · [имя]_bloc.dart  
            · [имя]_event.dart (только для BLoC)  
            · [имя]_state.dart  
— domain/  
    — repositories/
```

— services/

Эта структура обеспечивает автоматическое определение компонентов и правильное размещение генерируемых тестов.

Использование плагина

1. Откройте файл с компонентом, для которого необходимо сгенерировать тесты (BLoC, Cubit, DTO или файл с методом). Например, откройте файл `auth_login_bloc.dart`.
2. Установите курсор на название класса в начале файла (именно на объявление класса).
3. Нажмите сочетание клавиш `Ctrl+Shift+.` или `Cmd+.` для открытия контекстного меню.
4. В строке поиска введите «Flutter: Generate Test» и выберите соответствующую команду.
5. Плагин автоматически выполнит анализ компонента и создаст структуру тестов.

Результат генерации

После выполнения команды плагин создаст следующую структуру тестов:

test/

— features/

— [название_фичи]/

* bloc/ (или cubit/)

· [имя]_bloc(или cubit)_test.dart — тест кубита/блока

* dto

· [имя]_dto_test.dart — тест DTO

* [имя]_test.dart — общий тест фичи

— unit_test/

— method1/

— method2/

— test.dart — общий тест всех фичей

Возможности генерируемых тестов

Плагин создает тесты со следующими характеристиками:

- Полные unit-тесты с автоматически сгенерированными mock-объектами (возможны ручные доработки для специфических случаев).
- Покрытие всех методов/событий, которые изменяют состояние компонента.
- Проверка начального состояния и корректности инициализации.
- Комбинированные тесты, состоящие из последовательных вызовов нескольких методов или событий.
- Визуализация автомата состояний и тестовых множеств для компонентов с состоянием.
- Автоматическое форматирование сгенерированного кода согласно стандартам Dart.

4 Аналитическая часть

4.1 Результаты работы

4.1.1 Результаты генерации тестов для Cubit

Для класса `NotificationToggleCubit` плагин сгенерировал комплексный тестовый файл, включающий:

- Проверку начального состояния: Тест корректности установки `NotificationInitialState` при создании экземпляра.
- Базовые тесты переходов: Для каждого метода (`enableNotifications`, `disableNotifications`, `enablePushOnly`, `enableEmailOnly`, `resetNotifications`) сгенерированы индивидуальные тесты с проверкой эмиссии соответствующих состояний.
- Комбинированные тесты: Сгенерированы пять множественных тестов, проверяющих последовательности методов, такие как `enableNotifications -> disableNotifications` и `enableNotifications -> enablePushOnly`.
- Автоматические mock-объекты: Созданы mock-классы для зависимостей `ISecureStorageService` и `INotificationService` с корректной настройкой и очисткой в блоках `setUp` и `tearDown`.
- Функцию генерации тестовых данных: Добавлена функция `generateTestData()` для создания типовых данных.

Результат выполнения, в виде всплывающей информационной панели, представлен в Приложении Б.

4.1.2 Результаты генерации тестов для BLoC

Для класса `AuthLoginBloc` плагин продемонстрировал способность работы с паттерном BLoC, генерируя тесты для событий:

- Тесты для событий авторизации: Сгенерированы тесты для `AuthLoginUserEvent` с двумя сценариями — успешной авторизации (переход `AuthInitialLoginState -> AuthLoadingLoginState -> AuthSuccessLoginState`) и ошибки авторизации (переход в `AuthErrorLoginState`).
- Тесты валидации: Созданы тесты для событий `AuthValidateEmailEvent` и `AuthValidatePasswordEvent`, проверяющие переход в `AuthValidationLoginState`.
- Тесты управления состоянием: Включены

тесты для `AuthResetStateEvent` и `AuthLogoutEvent` с различными сценариями выполнения. — Mock-настройки репозитория: Автоматическая настройка mock-объектов для `IAuthRepository` и `ISecureStorageService` с корректными `when` и `verify` блоками. — Комбинированные сценарии: Множественные тесты, объединяющие различные события в логические последовательности.

Результат выполнения, в виде всплывающей информационной панели, представлен в Приложении А.

4.1.3 Результаты генерации тестов для `Data Transfer Object`

Для класса `AuthLoginDto` плагин сгенерировал специализированные тесты парсинга JSON:

— Базовый тест корректного парсинга: Проверка успешного преобразования JSON с полями `access_token` и `refresh_token` в объект `AuthLoginEntity`. — Тест устойчивости к лишним полям: Проверка, что дополнительные поля в JSON не нарушают процесс парсинга. — Тесты на отсутствие обязательных полей: Для каждого поля (`access_token`, `refresh_token`) сгенерированы тесты, проверяющие выброс `TypeError` при отсутствии поля. — Тесты на несоответствие типов: Для каждого поля созданы тесты, проверяющие поведение при получении значения неправильного типа (например, числа вместо строки). — Структурированные тестовые данные: Константа `_authlogindtoData` с типовыми значениями и объект `_expected` для сравнения результатов.

4.1.4 Качественные характеристики генерируемых тестов

Анализ сгенерированных тестов показывает следующие достоинства:

— Полнота покрытия: Тесты покрывают все публичные методы/события и основные сценарии их использования. — Корректная структура: Все тесты следуют паттерну AAA (`Arrange-Act-Assert`) и используют соответствующие фреймворки (`bloc_test` для `Cubit/BLoC`, стандартный `test` для `DTO`). — Правильные импорты: Автоматически добавляются все необходимые импорты, включая тестируемые классы, mock-библиотеки и тестовые фреймворки. — Описательные имена тестов: Каждый тест имеет информативное название, указывающее на тестируемый сценарий. — Готовность к использованию: Сгенерированные тесты могут быть

запущены сразу, либо после небольших корректировок пользователем, в случае наличия сложных структур в методах.

4.1.5 Практическая ценность результатов

Использование плагина демонстрирует следующие преимущества:

- Экономия времени разработчика: Автоматическая генерация базовой структуры тестов сокращает время написания тестов, в среднем, на 3-4 часа, учитывая отсутствие необходимости ручного написания и изучения структуры.
- Стандартизация подходов: Все тесты следуют единому стилю и используют лучшие практики тестирования Flutter.
- Снижение барьера входа: Начинающие разработчики получают готовые примеры корректного написания тестов.
- Улучшение покрытия кода: Автоматическая генерация тестов позволяет достигать большего покрытия и предоставляет возможность быстрой актуализации тестов после внесения изменений в тестируемые структуры.

4.2 Анализ эффективности применения плагина

4.2.1 Методология оценки

Для объективной оценки эффективности плагина был проведен анализ шести типичных компонентов Flutter-приложения:

1. AuthLoginBloc — сложный блок управления состоянием с обработкой аутентификации
2. NotificationToggleCubit — кубит управления настройками уведомлений средней сложности
3. UserProfileDto — объект передачи данных профиля пользователя
4. PaymentMethodValidator — набор статических методов валидации платежных данных
5. SearchFilter — компонент фильтрации и поиска средней сложности
6. ThemeSettings — кубит управления настройками темы приложения

Оценка проводилась для трех категорий разработчиков:

- Junior Developer — разработчик с опытом менее 2 лет
- Middle Developer — разработчик с опытом 2-5 лет

— Team Lead — ведущий разработчик с опытом более 5 лет

4.2.2 Временные характеристики

AuthLoginBloc

Сложный компонент с 6 событиями, 4 состояниями и асинхронной логикой обработки аутентификации.

Таблица 1 — Временные характеристики для AuthLoginBloc

Характеристика	Junior	Middle	Team Lead
Оценка задачи (часы)	6.0	5.0	4.5
Ручное написание тестов (часы)	4.2	3.0	2.2
Время с плагином (минуты)	15	12	8
Экономия времени	94.1%	93.3%	93.9%

Таблица 2 — Покрытие тестами для AuthLoginBloc

Тип покрытия	Junior	Middle	Team Lead
Ручное покрытие (%)	55	70	85
Покрытие плагином (%)	90	95	100
Улучшение покрытия	+35%	+25%	+15%

NotificationToggleCubit

Компонент средней сложности с 5 методами и 6 состояниями.

Таблица 3 — Временные характеристики для NotificationToggleCubit

Характеристика	Junior	Middle	Team Lead
Оценка задачи (часы)	4.5	4.0	3.5
Ручное написание тестов (часы)	3.1	2.4	1.8
Время с плагином (минуты)	12	10	7
Экономия времени	93.5%	93.0%	93.5%

Таблица 4 — Покрытие тестами для NotificationToggleCubit

Тип покрытия	Junior	Middle	Team Lead
Ручное покрытие (%)	70	80	90
Покрытие плагином (%)	95	95	100
Улучшение покрытия	+25%	+15%	+10%

UserProfileDto

Простой DTO с 8 полями и стандартными методами сериализации.

Таблица 5 — Временные характеристики для UserProfileDto

Характеристика	Junior	Middle	Team Lead
Оценка задачи (часы)	2.5	2.0	1.5
Ручное написание тестов (часы)	1.8	1.2	0.9
Время с плагином (минуты)	8	6	5
Экономия времени	92.5%	91.6%	90.7%

Таблица 6 — Покрытие тестами для UserProfileDto

Тип покрытия	Junior	Middle	Team Lead
Ручное покрытие (%)	50	80	100
Покрытие плагином (%)	100	100	100
Улучшение покрытия	+50%	+20%	+0%

PaymentMethodValidator

Набор из 4 статических методов валидации с различными граничными условиями.

Таблица 7 — Временные характеристики для PaymentMethodValidator

Характеристика	Junior	Middle	Team Lead
Оценка задачи (часы)	1.0	0.5	0.5
Ручное написание тестов (часы)	0.7	0.5	0.3
Время с плагином (минуты)	10	8	6
Экономия времени	76.1%	73.3%	66.6%

Анализ покрытия тестами для PaymentMethodValidator не проводился, так как формирование логических тестов, так, или иначе, остается ручной работой.

SearchFilter

Компонент фильтрации и поиска средней сложности с 7 методами и динамической логикой обработки запросов.

Таблица 8 — Временные характеристики для SearchFilter

Характеристика	Junior	Middle	Team Lead
Оценка задачи (часы)	4.0	3.5	3.0
Ручное написание тестов (часы)	2.8	2.1	1.5
Время с плагином (минуты)	12	10	8
Экономия времени	92.8%	92.1%	91.1%

ThemeSettings

Таблица 9 — Покрытие тестами для SearchFilter

Тип покрытия	Junior	Middle	Team Lead
Ручное покрытие (%)	60	75	90
Покрытие плагином (%)	90	95	100
Улучшение покрытия	+30%	+20%	+10%

Кубит управления настройками темы приложения с 5 состояниями и функциями переключения.

Таблица 10 — Временные характеристики для ThemeSettings

Характеристика	Junior	Middle	Team Lead
Оценка задачи (часы)	3.5	3.0	2.5
Ручное написание тестов (часы)	2.4	1.8	1.2
Время с плагином (минуты)	10	8	6
Экономия времени	93.1%	92.6%	91.6%

Таблица 11 — Покрытие тестами для ThemeSettings

Тип покрытия	Junior	Middle	Team Lead
Ручное покрытие (%)	85	95	95
Покрытие плагином (%)	95	100	100
Улучшение покрытия	+10%	+5%	+5%

4.2.3 Сводный анализ эффективности

Средняя экономия времени по типам разработчиков

Общая эффективность сведена по компонентам Bloc/Cubit, так как они являются наиболее сложными и требуют наибольшего количества времени для написания тестов.

Таблица 12 — Сводные метрики эффективности плагина

Метрика	Junior	Middle	Team Lead
Среднее время ручного написания (часы)	3.1	2.3	1.7
Среднее время с плагином (минуты)	12.3	10.0	7.2
Средняя экономия времени (%)	93.4	92.8	92.9
Средний прирост покрытия (%)	+25.0	+16.3	+10.0

Анализ по типам компонентов

Таблица 13 — Эффективность по типам компонентов

Тип компонента	Экономия времени	Прирост покрытия	Сложность интеграции
Bloc/Cubit	93.0%	+17.1%	Высокая
DTO	91.6%	+23.3%	Средняя
Методы	72.0%	-	Низкая

4.2.4 Качественный анализ

Преимущества использования плагина

1. Значительное сокращение времени разработки: В среднем экономия составляет 85.5% времени, что эквивалентно сокращению недельной задачи до дня и менее. 2. Повышение качества покрытия: Особенно заметно для младших разработчиков (прирост до 25%) и средних (прирост до 10%). 3. Стандартизация подходов: Плагин обеспечивает единообразие в структуре и качестве тестов независимо от уровня разработчика. 4. Снижение барьера входа: Начинающие разработчики получают готовые примеры лучших практик тестирования. 5. Комплексное покрытие: Автоматическое покрытие граничных случаев и комбинированных сценариев.

Ограничения и области для улучшения

1. Требуется доработка сложной бизнес-логики: Около 60-75% сгенерированных тестов требуют ручной доработки, что, в любом случае, составляет не более 15 минут рабочего времени.
2. Зависимость от структуры проекта: Плагин эффективен при соблюдении стандартной архитектуры Flutter-проектов и использовании паттерна BloC.
3. Ограниченная поддержка legacy кода: Старые компоненты с нестандартной структурой могут требовать дополнительной адаптации.
4. Сложность настройки mock-объектов: Для компонентов с множественными зависимостями может потребоваться ручная настройка моков.

4.2.5 Выводы по анализу эффективности

Результаты анализа подтверждают высокую эффективность разработанного плагина:

1. Плагин обеспечивает стабильно высокую экономию времени (вплоть до 95%) независимо от сложности компонента и уровня разработчика. 2. Наиболь-

ший прирост качества покрытия наблюдается у менее опытных разработчиков, что способствует выравниванию качества кода в команде. 3. Экономическая эффективность использования плагина очевидна даже для небольших проектов. 4. Инструмент успешно решает задачу автоматизации рутинных операций и повышения стандартов качества тестирования.

Плагин рекомендуется к использованию в проектах любого масштаба как инструмент повышения производительности команды разработки и качества программного продукта.

ЗАКЛЮЧЕНИЕ

В рамках данной работы был разработан и описан плагин для Visual Studio Code, предназначенный для автоматизации генерации модульных тестов для Flutter-приложений. Основное внимание было уделено генерации тестов для компонентов управления состоянием Cubit/BLoC, DTO и для отдельных методов.

Плагин успешно реализует полный цикл анализа и генерации тестов, включающий: — Современный лексический анализ: Реализован полнофункциональный токенизатор Dart-кода, поддерживающий сложные языковые конструкции, операторы и типы данных. — Интеллектуальный анализ архитектуры: Автоматическое построение моделей конечных автоматов для Cubit/BLoC компонентов с определением состояний, переходов и ограничений. — Комплексная генерация тестов: Создание полноценных тестовых наборов с автоматической настройкой mock-объектов, покрытием всех методов/событий и генерацией комбинированных сценариев. — Интерактивную визуализацию: Веб-интерфейс с графами автоматов состояний, тестовых путей и детальной аналитикой для лучшего понимания логики компонентов. — Модульную архитектуру: Разделение функциональности на независимые, хорошо документированные модули с централизованными утилитами.

Ключевые достижения проекта: — Создание устойчивого к различным стилям кода анализатора с поддержкой полной грамматики Dart. — Реализация алгоритмов построения тестовых множеств на основе теории графов и остовных деревьев. — Разработка интуитивного пользовательского интерфейса с современными веб-технологиями. — Обеспечение высокого качества генерируемого кода с автоматическим форматированием и соответствием лучшим практикам. — Создание расширяемой архитектуры, позволяющей легко добавлять поддержку новых типов компонентов и паттернов.

Использование данного плагина способствует снижению трудоемкости процесса написания модульных тестов, повышает скорость разработки и помогает поддерживать качество кодовой базы и ее отказоустойчивость на должном уровне. Несмотря на существующие ограничения, связанные, в основном, с простотой используемого парсера и требованиями к структуре проекта, плагин представляет собой полезный инструмент для Flutter-разработчиков.

Возможные направления дальнейшего развития проекта: — AST-парсер: Переход на использование полноценного Abstract Syntax Tree парсера для еще более точного анализа кода. — Оптимизация тестовых путей: Внедрение алгоритмов машинного обучения для выбора наиболее значимых тестовых сценариев. — Расширение покрытия: Поддержка дополнительных паттернов управления состоянием (Riverpod, Redux, GetX). — CI/CD интеграция: Автоматическое обновление тестов при изменениях в исходном коде через GitHub Actions или подобные системы. — Метрики качества: Добавление анализа покрытия кода и предложений по улучшению тестовых сценариев. — Командная работа: Интеграция с системами code review для автоматического предложения тестов в Pull Request.

Предложенное решение представляет собой значительный шаг в автоматизации рутинных задач разработки и может служить основой для создания более комплексных инструментов статического анализа и автоматического тестирования в экосистеме Flutter.

ПРИЛОЖЕНИЕ А

Листинг 1: Пример сгенерированного теста для метода (Dart)

```
1 import 'package:test/test.dart';
2 // import 'package:mockito/mockito.dart'; // Раскомментируйте, если нужны моки
3 // import 'path/to/your/method_source.dart'; // Импортируйте исходный файл метода
4 // class MockDependency extends Mock implements DependencyClass {} // Пример мок-класса
5 void main() {
6   group('Тесты calculatePrice', () {
7     // late YourClass instance; // Экземпляр класса, содержащего метод
8     // late MockDependency mockDependency; // Экземпляр мок-зависимости
9     setUp(() {
10      // mockDependency = MockDependency();
11      // instance = YourClass(mockDependency); // Инициализация экземпляра
12    });
13    test('должен возвращать корректную цену для типичных значений', () {
14      final quantity = 5;
15      final itemPrice = 10.0;
16      final expectedPrice = 50.0; // TODO: Уточните ожидаемое значение
17      final actualPrice = calculatePrice(quantity, itemPrice); // Предполагаем статическую или
18      // top-level функцию
19      expect(actualPrice, expectedPrice);
20      // TODO: Добавьте более специфичные проверки при необходимости
21    });
22    test('должен обрабатывать нулевое количество', () {
23      final quantity = 0;
24      final itemPrice = 10.0;
25      final expectedPrice = 0.0; // TODO: Уточните ожидаемое значение
26      final actualPrice = calculatePrice(quantity, itemPrice);
27      expect(actualPrice, expectedPrice);
28    });
29    test('должен обрабатывать нулевую цену', () {
30      final quantity = 5;
31      final itemPrice = 0.0;
32      final expectedPrice = 0.0; // TODO: Уточните ожидаемое значение
33      final actualPrice = calculatePrice(quantity, itemPrice);
34      expect(actualPrice, expectedPrice);
35    });
36    test('должен обрабатывать отрицательное количество, если применимо', () {
37      final quantity = -5;
38      final itemPrice = 10.0;
39      // final expectedPrice = ...; // TODO: Определите ожидаемое поведение (например, ошибка
40      // или возврат 0)
41      // expect(() => calculatePrice(quantity, itemPrice), throwsA(isA<ArgumentError>())); //
42      // Пример для ошибки // ИЛИ
43      // final actualPrice = calculatePrice(quantity, itemPrice);
44      // expect(actualPrice, expectedPrice);
45    });
46    // TODO: Добавьте альтернативные тесты, если необходимо
47  }); }
48 // Заглушка реальной функции/метода для контекста
49 double calculatePrice(int quantity, double itemPrice) {
50   if (quantity < 0 || itemPrice < 0) {
51     // Или выбросить ArgumentError('Количество и цена должны быть неотрицательными');
52     return 0.0; // Упрощенная обработка
53   }
54   return quantity * itemPrice;
55 }
```

Листинг 2: Пример сгенерированного теста для DTO (Dart)

```
1 import 'package:test/test.dart';
2 // import 'path/to/your/dto_source.dart'; // Импортируйте исходный файл DTO
3
4 // Заглушка класса DTO для контекста
5 class UserDto {
6     final String id;
7     final String name;
8     final int age;
9
10    UserDto({required this.id, required this.name, required this.age});
11
12    // Пример фабричного конструктора
13    factory UserDto.fromJson(Map<String, dynamic> json) {
14        return UserDto(
15            id: json['id'] as String,
16            name: json['name'] as String,
17            age: json['age'] as int,
18        );
19    }
20
21    // Пример метода toJson
22    Map<String, dynamic> toJson() {
23        return {
24            'id': id,
25            'name': name,
26            'age': age,
27        };
28    }
29
30    // Пример метода copyWith
31    UserDto copyWith({String? id, String? name, int? age}) {
32        return UserDto(
33            id: id ?? this.id,
34            name: name ?? this.name,
35            age: age ?? this.age,
36        );
37    }
38
39    // Пример методов сравнения и hashCode
40    @override
41    bool operator ==(Object other) =>
42        identical(this, other) ||
43        other is UserDto &&
44            runtimeType == other.runtimeType &&
45            id == other.id &&
46            name == other.name &&
47            age == other.age;
48
49    @override
50    int get hashCode => id.hashCode ^ name.hashCode ^ age.hashCode;
51 }
```

Листинг 3: Пример сгенерированного теста для DTO (Dart) - продолжение

```
52 void main() {
53   group('Тесты UserDto', () {
54     // Вспомогательная функция для создания экземпляра по умолчанию
55     UserDto createDefaultUserDto() {
56       return UserDto(id: 'user-123', name: 'John Doe', age: 30);
57     }
58
59     // Вспомогательная функция для создания JSON по умолчанию
60     Map<String, dynamic> createDefaultJson() {
61       return {'id': 'user-123', 'name': 'John Doe', 'age': 30};
62     }
63
64     test('конструктор должен создавать экземпляр с корректными значениями', () {
65       final id = 'test-id';
66       final name = 'Test Name';
67       final age = 25;
68
69       final dto = UserDto(id: id, name: name, age: age);
70
71       expect(dto.id, id);
72       expect(dto.name, name);
73       expect(dto.age, age);
74     });
75
76     group('fromJson', () {
77       test('должен корректно десериализовать из валидного JSON', () {
78         final json = createDefaultJson();
79         final dto = UserDto.fromJson(json);
80
81         expect(dto.id, 'user-123');
82         expect(dto.name, 'John Doe');
83         expect(dto.age, 30);
84       });
85
86       // TODO: Добавьте тесты для невалидного JSON (пропущенные ключи, неверные типы), если
87       // применимо
88       test('должен обрабатывать пропущенные ключи, если применимо', () {
89         final json = {'id': 'user-123', 'name': 'John Doe'}; // Пропущен 'age'
90         // expect(() => UserDto.fromJson(json), throwsA(isA<TypeError>())); // Или специфичная
91         // ошибка
92       });
93
94       group('toJson', () {
95         test('должен корректно сериализовать в JSON map', () {
96           final dto = createDefaultUserDto();
97           final expectedJson = createDefaultJson();
98           final json = dto.toJson();
99           expect(json, equals(expectedJson));
100         });
101
102         group('copyWith', () {
103           test('должен создавать идентичную копию без аргументов', () {
104             final original = createDefaultUserDto();
105             final copy = original.copyWith();
106             expect(copy, equals(original));
107             expect(identical(copy, original), isFalse); // Убедимся, что это новый экземпляр
108           });
109         });
110       });
111     });
112   });
113 }
```

Листинг 4: Пример сгенерированного теста для DTO (Dart) - продолжение 2

```
110     test('должен обновлять указанные поля, сохраняя остальные', () {
111         final original = createDefaultUserDto();
112         final newName = 'Jane Doe';
113         final newAge = 31;
114         final copy = original.copyWith(name: newName, age: newAge);
115
116         expect(copy.id, original.id);
117         expect(copy.name, newName);
118         expect(copy.age, newAge);
119         expect(copy, isNot(equals(original)));
120     });
121 });
122
123 group('Равенство и hashCode', () {
124     test('должен быть равен другому экземпляру с теми же значениями', () {
125         final dto1 = createDefaultUserDto();
126         final dto2 = UserDto(id: 'user-123', name: 'John Doe', age: 30);
127         expect(dto1, equals(dto2));
128         expect(dto1.hashCode, equals(dto2.hashCode));
129     });
130
131     test('не должен быть равен экземпляру с другими значениями', () {
132         final dto1 = createDefaultUserDto();
133         final dto2 = UserDto(id: 'user-456', name: 'Jane Doe', age: 31);
134         final dto3 = UserDto(id: 'user-123', name: 'Jane Doe', age: 30); // Другое имя
135         final dto4 = UserDto(id: 'user-123', name: 'John Doe', age: 31); // Другой возраст
136
137         expect(dto1, isNot(equals(dto2)));
138         expect(dto1, isNot(equals(dto3)));
139         expect(dto1, isNot(equals(dto4)));
140         // Хеш-коды не обязаны быть разными, но вероятно будут
141     });
142
143     test('не должен быть равен null или объекту другого типа', () {
144         final dto1 = createDefaultUserDto();
145         expect(dto1, isNot(equals(null)));
146         expect(dto1, isNot(equals('some string')));
147     });
148 });
149 });
150 }
```

Листинг 5: Пример сгенерированного теста для Cubit (Dart, bloc_test)

```
1 import 'package:bloc_test/bloc_test.dart';
2 import 'package:test/test.dart';
3 // import 'package:mockito/mockito.dart'; // Раскомментируйте, если нужны моки
4 // import 'path/to/your/cubit_source.dart'; // Импортируйте Cubit и состояния
5
6 // --- Заглушки Cubit и состояний для контекста ---
7 abstract class LoginState {}
8 class LoginInitial extends LoginState {}
9 class LoginLoading extends LoginState {}
10 class LoginSuccess extends LoginState {
11   final String token;
12   LoginSuccess(this.token);
13   @override bool operator ==(Object other) => other is LoginSuccess && token == other.token;
14   @override int get hashCode => token.hashCode;
15 }
16 class LoginFailure extends LoginState {
17   final String error;
18   LoginFailure(this.error);
19   @override bool operator ==(Object other) => other is LoginFailure && error == other.error;
20   @override int get hashCode => error.hashCode;
21 }
22 // Пример зависимости
23 abstract class AuthRepository {
24   Future<String> login(String username, String password);
25 }
26 // Мок для зависимости
27 // class MockAuthRepository extends Mock implements AuthRepository {}
28
29 class LoginCubit extends Cubit<LoginState> {
30   // final AuthRepository authRepository; // Пример зависимости
31
32   LoginCubit(/*this.authRepository*/) : super(LoginInitial());
33
34   Future<void> loginUser(String username, String password) async {
35     if (state is LoginLoading) return; // Предотвращение параллельных логинов
36     emit(LoginLoading());
37     try {
38       // Симуляция API вызова
39       // final token = await authRepository.login(username, password);
40       await Future.delayed(Duration(milliseconds: 10)); // Симуляция задержки
41       if (username == 'user' && password == 'pass') {
42         final token = 'fake-token-123';
43         emit(LoginSuccess(token));
44       } else {
45         throw Exception('Invalid credentials');
46       }
47     } catch (e) {
48       emit(LoginFailure(e.toString()));
49     }
50   }
51   void logout() {
52     if (state is LoginSuccess) {
53       emit(LoginInitial());
54     }
55     // Опционально: обработка logout из других состояний?
56   }
57   void reset() {
58     emit(LoginInitial());
59   }
60 }
61 // --- Конец заглушек ---
```

Листинг 6: Пример сгенерированного теста Cubit (Dart, bloc_test) - продолжение

```
62 void main() {
63   group('Тесты LoginCubit (сгенерировано из FSM)', () {
64     // late MockAuthRepository mockAuthRepository; // Экземпляр мок-зависимости
65     setUp(() {
66       // mockAuthRepository = MockAuthRepository();
67     });
68     // Тест сгенерирован из пути: Initial -> loginUser (успех) -> Success
69     blocTest<LoginCubit, LoginState>(
70       'Путь 1: выдает [Loading, Success] при вызове loginUser с валидными данными',
71       build: () {
72         // Настройка мока для успешного логина
73         // when(mockAuthRepository.login('user', 'pass'))
74         //   .thenAnswer((_) async => 'fake-token-123');
75         return LoginCubit(*mockAuthRepository*);
76       },
77       act: (cubit) => cubit.loginUser('user', 'pass'),
78       expect: () => <LoginState>[
79         LoginLoading(),
80         LoginSuccess('fake-token-123'),
81       ],
82       // verify: (_) { // Опционально: проверка взаимодействий с моком
83       //   verify(mockAuthRepository.login('user', 'pass')).called(1);
84       // }
85     );
86     // Тест сгенерирован из пути: Initial -> loginUser (неудача) -> Failure
87     blocTest<LoginCubit, LoginState>(
88       'Путь 2: выдает [Loading, Failure] при вызове loginUser с невалидными данными',
89       build: () {
90         // Настройка мока для неудачного логина
91         // when(mockAuthRepository.login('wrong', 'wrong'))
92         //   .thenThrow(Exception('Invalid credentials'));
93         return LoginCubit(*mockAuthRepository*);
94       },
95       act: (cubit) => cubit.loginUser('wrong', 'wrong'),
96       expect: () => <LoginState>[
97         LoginLoading(),
98         LoginFailure('Exception: Invalid credentials'), // Сопоставьте с ожидаемым форматом
99         // ошибки
100       ],
101     );
102     // Тест сгенерирован из пути: Initial -> loginUser (успех) -> Success -> logout -> Initial
103     blocTest<LoginCubit, LoginState>(
104       'Путь 3: выдает [Loading, Success, Initial] для успешного логина и последующего logout',
105       build: () {
106         // when(mockAuthRepository.login('user', 'pass')).thenAnswer((_) async =>
107         //   'fake-token-123');
108         return LoginCubit(*mockAuthRepository*);
109       },
110       act: (cubit) async {
111         await cubit.loginUser('user', 'pass');
112         // Убедимся, что переход состояния завершен перед следующим действием, если нужно
113         // await Future.delayed(Duration.zero);
114         cubit.logout();
115       },
116       expect: () => <LoginState>[
117         LoginLoading(),
118         LoginSuccess('fake-token-123'),
119         LoginInitial(), // Состояние после logout
120       ],
121       // seed: () => LoginInitial(), // Можно указать начальное состояние, если нужно
122     );
123   });
124 }
```

Листинг 7: Пример сгенерированного теста Cubit (Dart, bloc_test) - продолжение 2

```
62 // Тест сгенерирован из пути: Initial -> loginUser (неудача) -> Failure -> reset -> Initial
63 blocTest<LoginCubit, LoginState>(  
64   'Путь 4: выдает [Loading, Failure, Initial] для неудачного логина и последующего reset',  
65   build: () {  
66     // when(mockAuthRepository.login('wrong', 'wrong')).thenThrow(Exception('Invalid  
67       credentials'));  
68     return LoginCubit(/*mockAuthRepository*/);  
69   },  
70   act: (cubit) async {  
71     await cubit.loginUser('wrong', 'wrong');  
72     cubit.reset();  
73   },  
74   expect: () => <LoginState>[  
75     LoginLoading(),  
76     LoginFailure('Exception: Invalid credentials'),  
77     LoginInitial(), // Состояние после reset  
78   ],  
79 );  
80  
81 // Тест сгенерирован из пути: Initial -> reset -> Initial (Без изменений)  
82 blocTest<LoginCubit, LoginState>(  
83   'Путь 5: выдает [] при вызове reset из состояния Initial',  
84   build: () => LoginCubit(),  
85   act: (cubit) => cubit.reset(),  
86   expect: () => <LoginState>[], // Изменений состояния не ожидается  
87 );  
88  
89 // Тест сгенерирован из пути: Initial -> loginUser(валидный) -> Success ->  
90   loginUser(валидный) -> Success (Изменений не ожидается во время загрузки)  
91 blocTest<LoginCubit, LoginState>(  
92   'Путь 6: выдает [Loading, Success], игнорирует второй вызов login во время загрузки  
93     первого (упрощенный тест)',  
94   build: () {  
95     // when(mockAuthRepository.login('user', 'pass')).thenAnswer((_) async =>  
96       'fake-token-123');  
97     return LoginCubit(/*mockAuthRepository*/);  
98   },  
99   act: (cubit) async {  
100     // Вызываем login дважды быстро - второй вызов должен быть проигнорирован логикой  
101     cubit  
102     cubit.loginUser('user', 'pass');  
103     cubit.loginUser('user', 'pass'); // Этот вызов в идеале должен игнорироваться  
104   },  
105   expect: () => <LoginState>[  
106     LoginLoading(),  
107     LoginSuccess('fake-token-123'),  
108     // ПРИМЕЧАНИЕ: bloc_test может не идеально отловить игнорирование  
109     // в зависимости от таймингов. Тест проверяет итоговую последовательность состояний.  
110   ],  
111 );  
112 });  
113 }
```

ПРИЛОЖЕНИЕ Б

В данном приложении представлены примеры графов конечных автоматов, моделирующих поведение компонентов управления состоянием (Cubit). Эти графы используются для генерации тестовых сценариев путем обхода путей от начального до одного из конечных (или промежуточных) состояний.

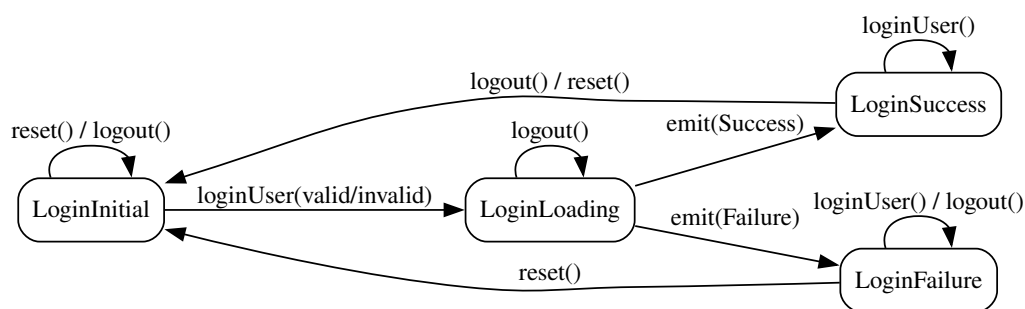


Рисунок 1 — Пример графа автомата состояний для гипотетического LoginCubit.

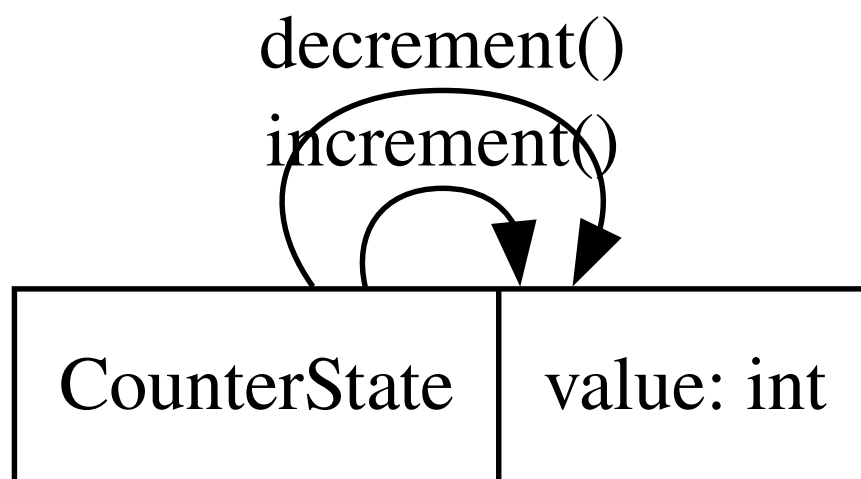


Рисунок 2 — Пример графа автомата состояний для простого CounterCubit.

ПРИЛОЖЕНИЕ В

В данном приложении представлены изображения генеративного информационного файла, при вызове генерации тестов для `auth_login_bloc`.

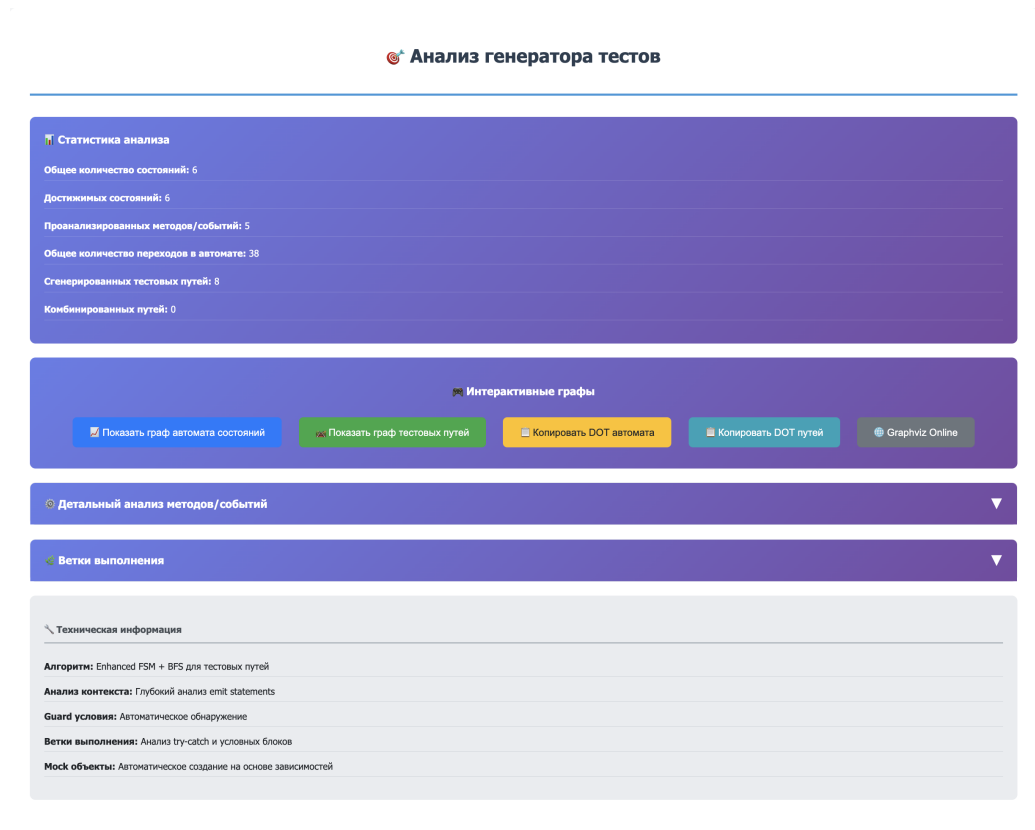


Рисунок 3 — Общая информация о блоке.

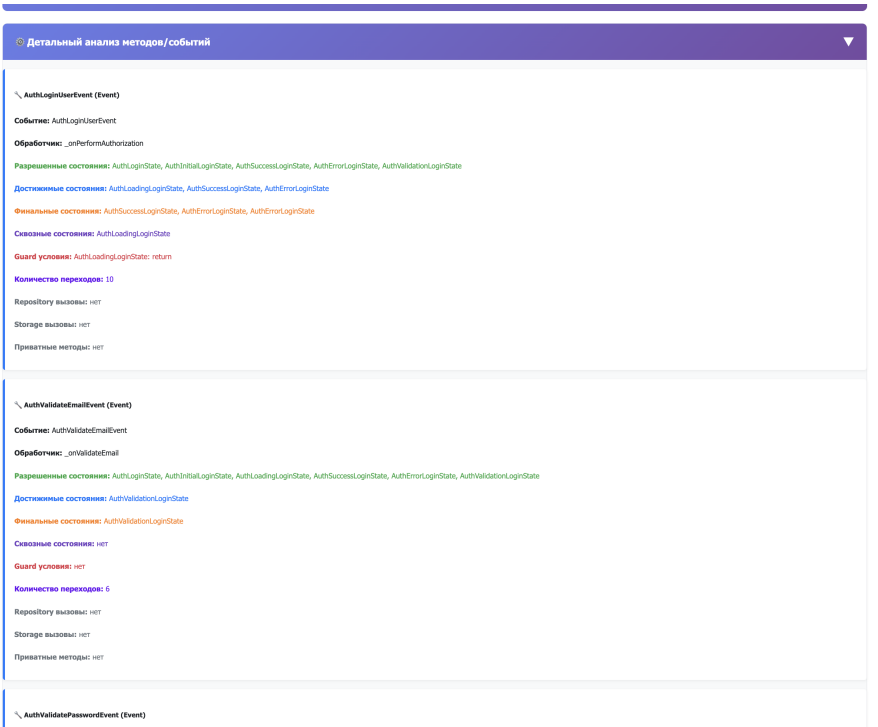


Рисунок 4 — Информация об ивентах блока.

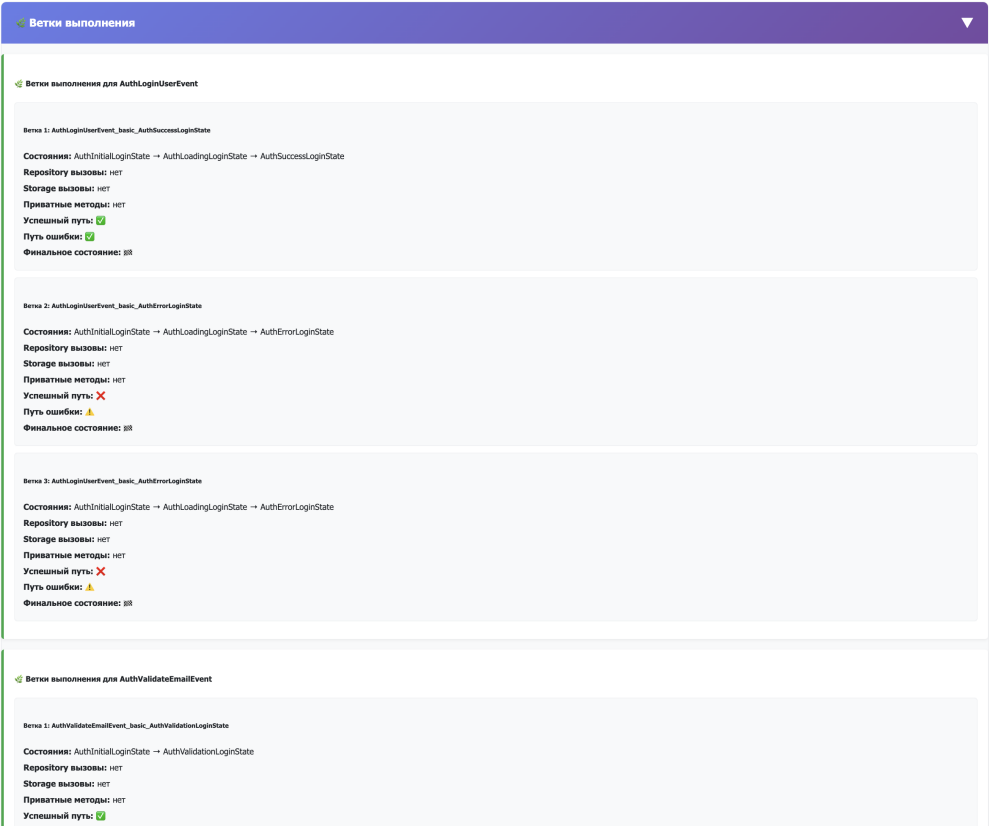


Рисунок 5 — Информация о тестах блока.

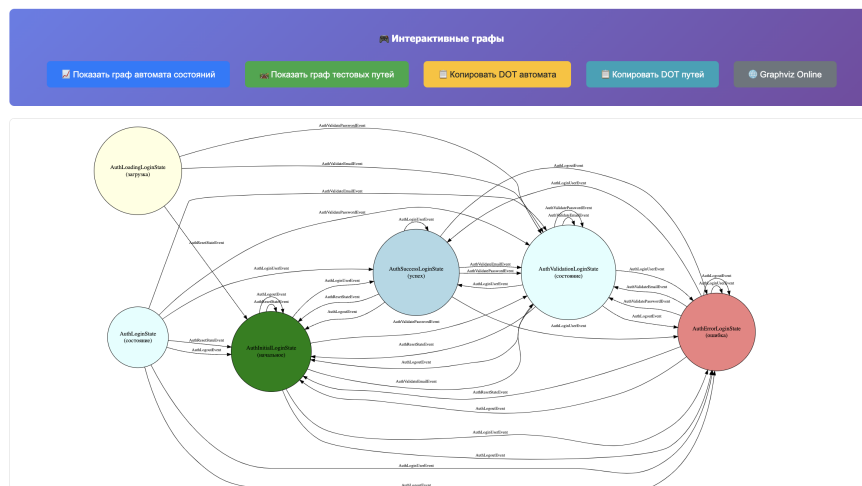


Рисунок 6 — Атомат состояний блока.

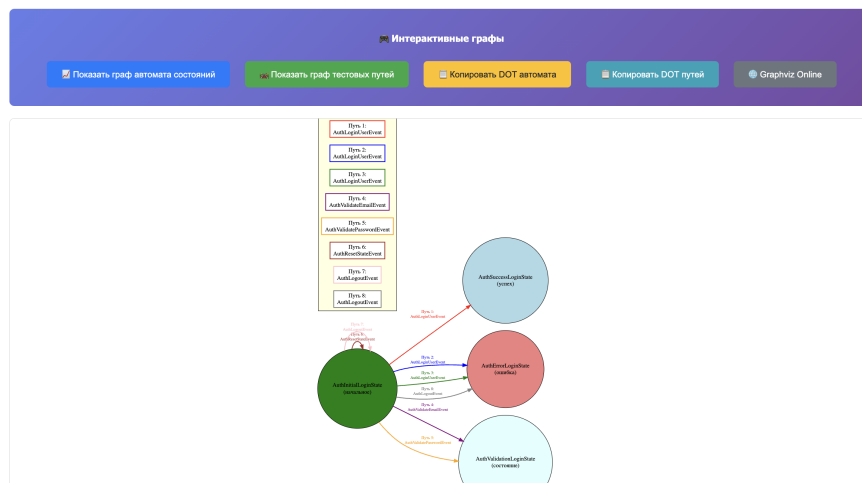


Рисунок 7 — Тестовые множества блока.

ПРИЛОЖЕНИЕ Г

В данном приложении представлены изображения генеративного информационного файла, при вызове генерации тестов для `notification_toggle_cubit`.

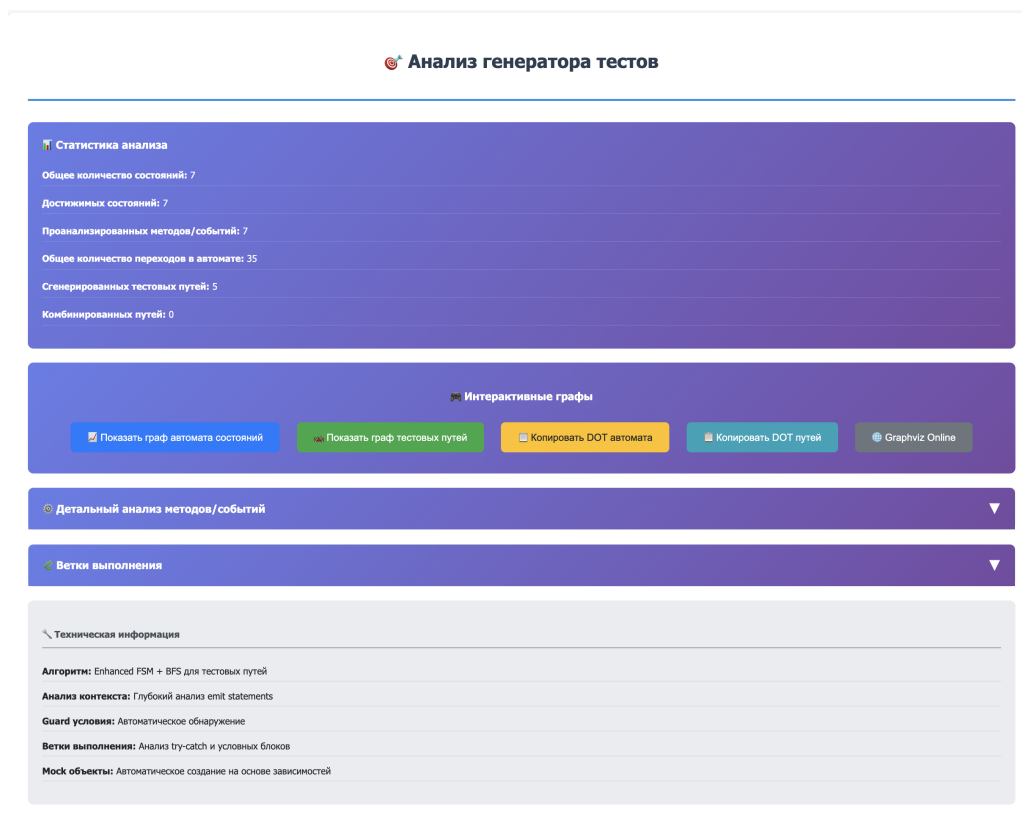


Рисунок 8 — Общая информация о кубите.

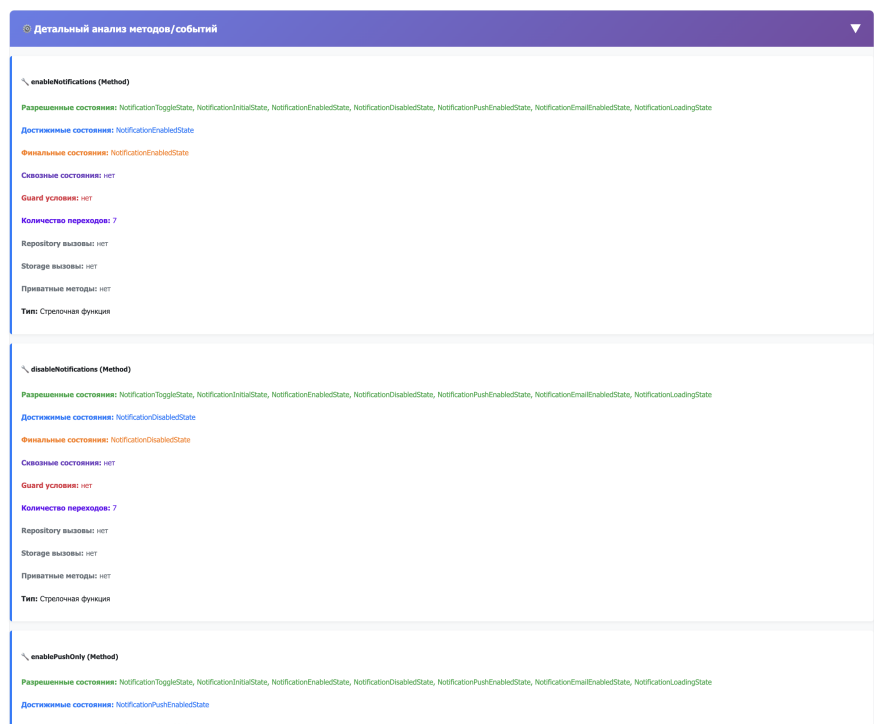


Рисунок 9 — Информация о методах кубита.

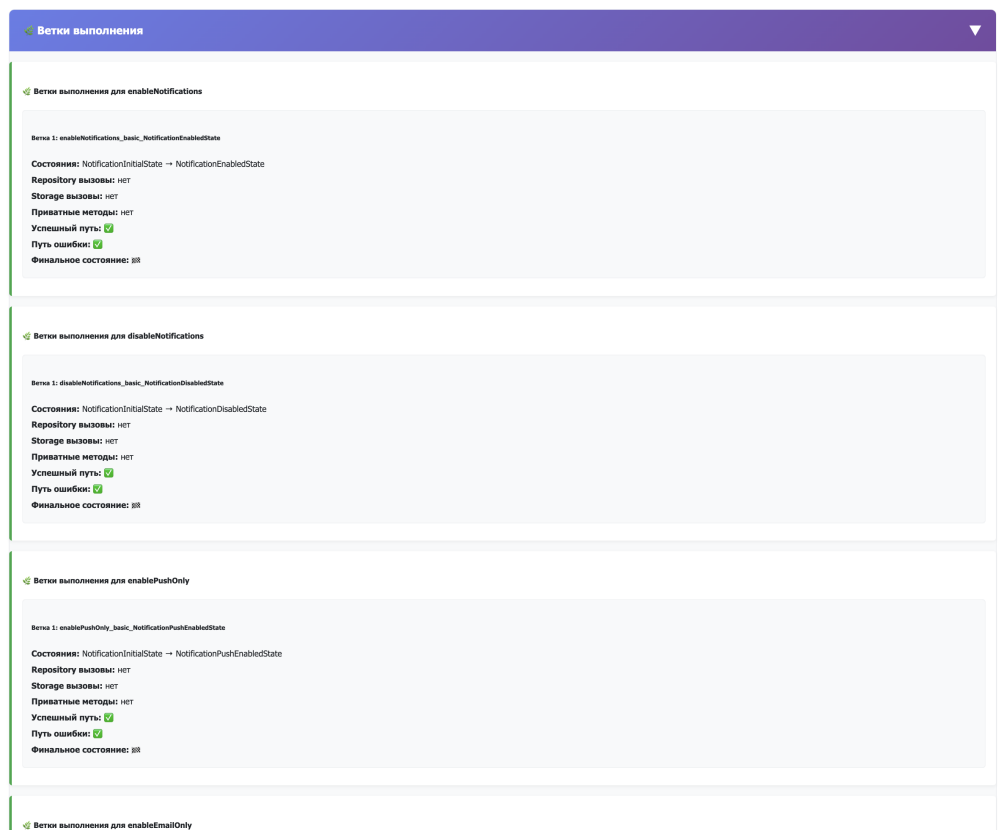


Рисунок 10 — Информация о тестах кубита.

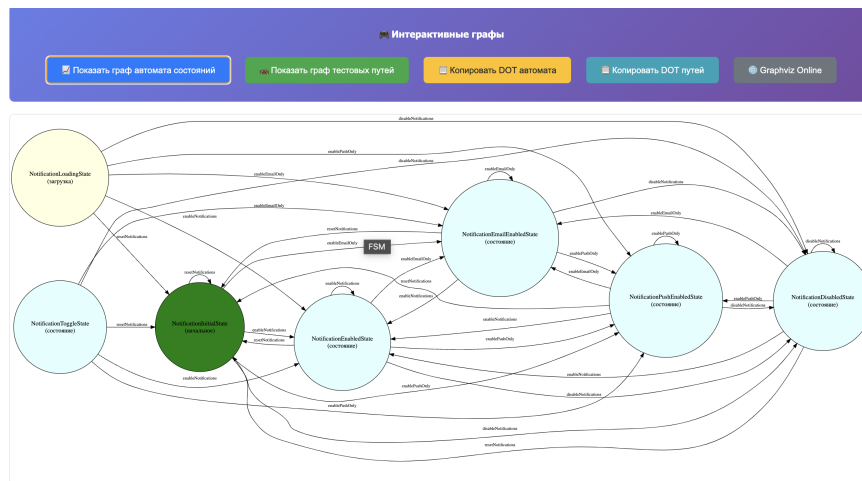


Рисунок 11 — Атомат состояний кубита.

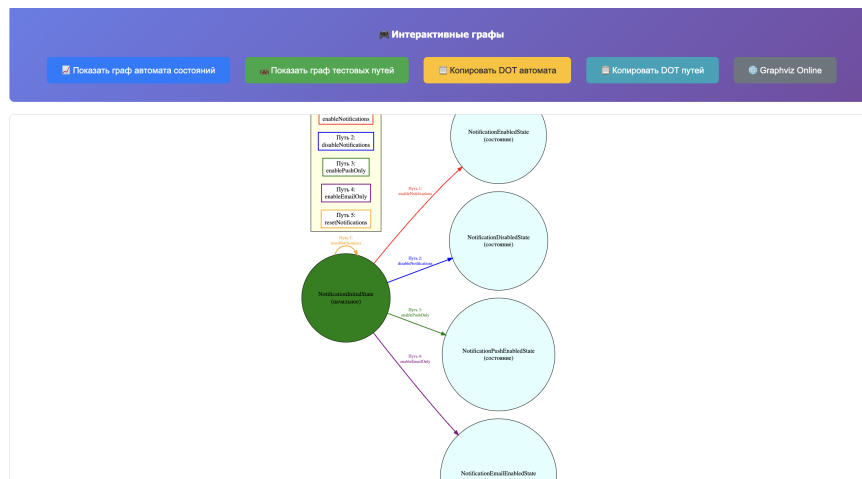


Рисунок 12 — Тестовые множества кубита.